

SC2001 Algorithm Design & Analysis
NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-23 — Generated for bumbsclaw/ntu-cs-textbooks

Contents

1	Foundations: From Zero to Algorithmic Reasoning	1
1.1	Discrete Mathematics Refresher	1
1.1.1	Sets, Functions, Relations	2
1.1.2	Logic and Quantifiers	2
1.1.3	Sums, Products, Logarithms	2
1.2	Proof Techniques	3
1.2.1	Direct Proof	4
1.2.2	Contrapositive	4
1.2.3	Proof by Contradiction	4
1.2.4	Counterexample and Cases	4
1.3	Mathematical Induction	4
1.3.1	Weak Induction	4
1.3.2	Strong Induction	5
1.3.3	Structural Induction	5
1.4	What Is an Algorithm?	5
1.4.1	Pseudocode Conventions	5
1.5	The Random Access Machine (RAM) Model	6
1.5.1	Cost Model and Word-RAM Refinement	6
1.6	Correctness: Loop Invariants and Termination	6
1.7	Chapter Notes	7
1.8	Exercises	7
2	Asymptotic Analysis	9

2.1	Why Asymptotics?	9
2.2	The Five Notations	10
2.3	The Limit Method	11
2.4	Growth Hierarchy	11
2.5	Best, Worst, and Average Case	12
2.6	Bridge to Recurrences	12
2.7	Chapter Notes	13
2.8	Exercises	13
3	Recurrences	14
3.1	From Algorithms to Recurrences	14
3.2	The Substitution Method	15
3.3	The Recursion-Tree Method	15
3.4	The Master Theorem	16
3.4.1	Worked Applications	17
3.5	Beyond Equal Subproblems: Akra–Bazzi	18
3.6	Chapter Notes and Exercises	19
4	Divide and Conquer	20
4.1	The Divide-and-Conquer Paradigm	20
4.2	MergeSort	21
4.3	QuickSort	21
4.4	Closest Pair of Points	22
4.5	Strassen’s Matrix Multiplication	23
4.6	Exercises	24
5	Greedy Algorithms	25
5.1	The Greedy Paradigm and the Exchange Argument	25
5.2	Interval Scheduling: Earliest Finish Time	26
5.3	Huffman Coding	27
5.4	Minimum Spanning Trees: The Cut Property	28

5.5	When Greedy Fails: Coin Change	29
5.6	Exercises	30
6	Dynamic Programming	31
6.1	The DP Paradigm	31
6.1.1	Optimal Substructure	31
6.1.2	Overlapping Subproblems	31
6.1.3	Memoisation versus Tabulation	32
6.2	Longest Common Subsequence	32
6.2.1	Optimal Substructure and Recurrence	32
6.2.2	Tabulation and Walkthrough	33
6.3	0/1 Knapsack	33
6.3.1	Table and Backtracking	34
6.4	Matrix Chain Multiplication	34
6.5	Coin Change: Unbounded Knapsack	35
6.6	Exercises	36
7	Graph Traversal, Shortest Paths, and String Matching	37
7.1	Graph Traversal: BFS and DFS	37
7.1.1	BFS: Queue and Layering	38
7.1.2	DFS: Stack and Structure	38
7.2	Shortest Paths	39
7.2.1	Dijkstra: Greedy with a Heap	39
7.2.2	Bellman–Ford: Relaxation and Negative Cycles	40
7.2.3	Floyd–Warshall: DP over Intermediate Vertices	41
7.3	String Matching	41
7.3.1	Naïve $\Theta((n - m + 1)m)$ Scan	41
7.3.2	KMP: Prefix Function and Automaton	41
7.3.3	Rabin–Karp: Rolling Hash	42
7.4	Maximum Flow: Ford–Fulkerson and Min Cut	42
7.5	Chapter Notes	43

7.6 Exercises	43
8 NP-Completeness	45
8.1 P, NP, and coNP	45
8.2 Polynomial Reductions and NP-Completeness	46
8.2.1 SAT $\leq_p$ 3SAT: clause splitting	46
8.2.2 3SAT $\leq_p$ CLIQUE: triangles and conflicts	47
8.2.3 CLIQUE $\leq_p$ VERTEX COVER: complement	47
8.3 Coping with NP-Completeness	47
8.3.1 2-approximation for VERTEX COVER via maximal matching	48
8.3.2 Greedy SET COVER: $\ln n$ approximation	48
8.3.3 Heuristic for TSP: nearest neighbour	48
8.4 P vs. NP and Why P $\neq$ NP Is Believed	49
8.5 Chapter Notes	49
8.6 Exercises	49

Chapter 1

Foundations: From Zero to Algorithmic Reasoning

“An algorithm must be seen to be believed.” — Donald Knuth. Before we design faster algorithms, we must agree on what an algorithm *is*, how to argue that it is *correct*, and what it costs to run. This chapter builds that floor from first principles.

From zero: we build here, no MH1812/SC1007 needed. This chapter defines every discrete-math notion from scratch—sets, functions, logic, sums, proofs, induction—with pictures before formalism. No prior discrete math or data-structures course is assumed. Later chapters depend only on what is built here.

Learning Outcomes

After this chapter you will be able to:

- (L1) recall essential discrete-mathematical notation and handle sums, logarithms, floors and ceilings fluently;
- (L2) distinguish and apply direct proof, contrapositive, contradiction, and counterexamples;
- (L3) prove statements by (weak and strong) mathematical induction;
- (L4) define *algorithm* precisely and describe the RAM model and its cost measure;
- (L5) prove correctness of iterative algorithms using loop invariants and termination arguments.

1.1 Discrete Mathematics Refresher

We collect notation used throughout the book—built from zero, not assumed from MH1812. Each idea appears as picture $\rightarrow$ words $\rightarrow$ definition.

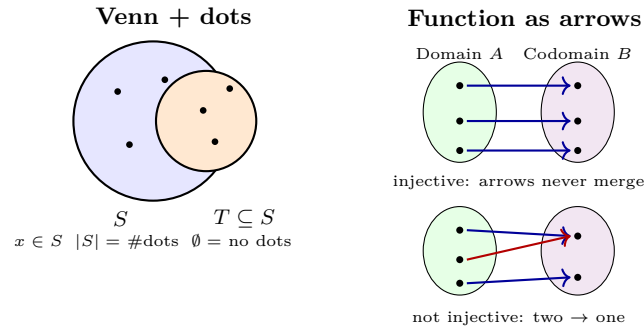


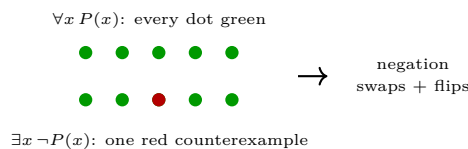
Figure 1.1: Left: sets as dot collections inside Venn ovals. Right: a function is arrows domain $\rightarrow$ codomain; injective means no two arrows share a target.

1.1.1 Sets, Functions, Relations

A *set* is an unordered collection of distinct objects. We write $x \in S$, $S \subseteq T$, $S \subsetneq T$, $|S|$ for cardinality, $\emptyset$ for the empty set, $\mathbb{N} = \{0, 1, 2, \dots\}$ (we include 0), $\mathbb{Z}, \mathbb{Q}, \mathbb{R}, \mathbb{Z}^+$. A *function* $f : A \rightarrow B$ assigns to each $a \in A$ exactly one $b = f(a) \in B$. A is the domain, B the codomain, $f(A) = \{f(a) : a \in A\}$ the image. f is *injective* if $f(a) = f(a') \Rightarrow a = a'$, *surjective* if $f(A) = B$, *bijective* if both. Composition $(g \circ f)(a) = g(f(a))$. A *relation* $R \subseteq A \times A$ may be reflexive, symmetric, transitive, antisymmetric—yielding equivalences and partial orders. A *graph* $G = (V, E)$ with $E \subseteq V \times V$ is used from Ch. 6; we define paths/cycles/trees when first needed (no SC1007 assumed).

1.1.2 Logic and Quantifiers

Everyday first. “Every student passed” vs. “Some student passed” already contain the two quantifiers. Picture a row of dots (students); colour a dot green if it satisfies property P .



Formally, propositions combine via $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$. Quantifiers $\forall$ (*for all*) and $\exists$ (*there exists*) bind variables: $\forall n \in \mathbb{N}, P(n)$ and $\exists x \in S, Q(x)$. Swapping gives De Morgan:

$$\neg(\forall x P(x)) \equiv \exists x \neg P(x), \quad \neg(\exists x P(x)) \equiv \forall x \neg P(x).$$

“Not every student passed” $\equiv$ “Some student failed”—swap $\forall \leftrightarrow \exists$ and flip $P \rightarrow \neg P$. Implication $P \Rightarrow Q \equiv \neg P \vee Q \equiv (\neg Q \Rightarrow \neg P)$ (contrapositive).

1.1.3 Sums, Products, Logarithms

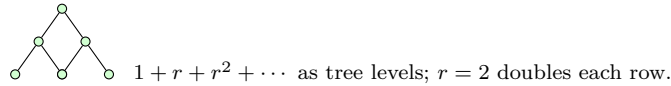
We build each closed form from a picture, then state it. Each box is a 3-line derivation.

Arithmetic $1 + 2 + \cdots + n = n(n+1)/2$ (**Gauss pairing**). Pair $1 + n = n + 1$, $2 + (n-1) = n + 1$, $\dots$ — $n/2$ pairs. Two staircases make an $n \times (n+1)$ rectangle.



$$1 + \cdots + n \text{ (two copies)} = n(n+1) \text{ dots} \quad \sum_{i=1}^n i = n(n+1)/2.$$

Geometric $1 + r + \cdots + r^{n-1} = (r^n - 1)/(r - 1)$ (**binary tree**). Level i has r^i nodes; stack levels form a tree. Multiply by $(r - 1)$ telescopes. If $|r| < 1$, $\sum_{i \geq 0} r^i = 1/(1 - r)$.

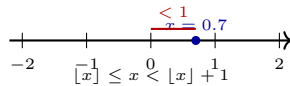


$$\sum_{i=0}^{n-1} r^i = (r^n - 1)/(r - 1).$$

Harmonic $H_n = \sum_{i=1}^n 1/i = \ln n + \gamma + O(1/n)$ (**coupon/stack**). H_n is area of n bars width 1, height $1/i$ —sandwiched between $\int_1^{n+1} dx/x$ and $1 + \int_1^n dx/x$. Hence $\ln(n+1) \leq H_n \leq 1 + \ln n$; limit $\gamma \approx 0.5772$.

Logarithms (rulers). $\log$ without base $= \log_2$ for algorithms; $\ln = \log_e$, $\lg = \log_2$ always. Change of base is rescaling a ruler: $\log_a n = (\log_b n)/(\log_b a) = \Theta(\log_b n)$ —base irrelevant inside $O(\cdot)$.

Floors/ceilings (number line, off-by-one ≤ 1).



$\lfloor x \rfloor \leq x < \lfloor x \rfloor + 1$, $\lceil x \rceil - 1 < x \leq \lceil x \rceil$, $x - 1 < \lfloor x \rfloor \leq x \leq \lceil x \rceil < x + 1$ —dropping $\lfloor \cdot \rfloor$ errs by at most 1. Also $n! \leq n^n$, $2^n \leq n!$ for $n \geq 4$, $(1 + 1/n)^n < e < (1 + 1/n)^{n+1}$.

Example 1.1 (Manipulating sums). For $n \geq 1$, $\sum_{i=1}^n (2i - 1) = 2 \frac{n(n+1)}{2} - n = n^2$ by linearity $\sum ca_i = c \sum a_i$, $\sum (a_i + b_i) = \sum a_i + \sum b_i$.

1.2 Proof Techniques

Every correctness proof uses one of four patterns.

Definition 1.1 (Theorem, Lemma, Corollary). A *theorem* is a statement proved true. A *lemma* is a helper theorem; a *corollary* is an immediate consequence.

1.2.1 Direct Proof

Assume hypothesis; deduce conclusion.

Example 1.2. If n odd then n^2 odd. *Proof.* $n = 2k + 1 \Rightarrow n^2 = 4k^2 + 4k + 1 = 2(2k^2 + 2k) + 1$ odd. $\square$

1.2.2 Contrapositive

Prove $\neg Q \Rightarrow \neg P$ instead of $P \Rightarrow Q$.

Example 1.3. If n^2 even then n even. Contrapositive: if n odd then n^2 odd—proved above. $\square$

1.2.3 Proof by Contradiction

Assume false, derive $R \wedge \neg R$.

Example 1.4 (Irrationality of $\sqrt{2}$). If $\sqrt{2} = p/q$ lowest terms, $2q^2 = p^2 \Rightarrow p$ even $= 2k \Rightarrow q^2 = 2k^2 \Rightarrow q$ even—contradicts $\gcd(p, q) = 1$. $\square$

1.2.4 Counterexample and Cases

To disprove $\forall x P(x)$ give one x with $\neg P(x)$; to prove by cases exhaust all possibilities.

Example 1.5. “Every $n > 1$ prime” false: $4 = 2 \cdot 2$. “ $n^2 \bmod 4 \in \{0, 1\}$ ”: even $n = 2k \Rightarrow 4k^2 \equiv 0$; odd $n = 2k + 1 \Rightarrow 4k(k + 1) + 1 \equiv 1$.

Remark 1.1 (How to choose). Try direct first. If conclusion is a negation (“not”, “irrational”, “no algorithm”), try contradiction. If P itself is negated, contrapositive is often direct in disguise.

1.3 Mathematical Induction

Induction proves loop invariants, recurrence closed forms, and properties of recursive structures.

1.3.1 Weak Induction

To prove $\forall n \geq n_0, P(n)$: (I1) base $P(n_0)$; (I2) $\forall k \geq n_0, P(k) \Rightarrow P(k + 1)$. $P(k)$ is the IH.

Theorem 1.1 (Sum of first n integers). For $n \geq 1$, $\sum_{i=1}^n i = n(n + 1)/2$.

Proof. Base $1 = 1 \cdot 2/2$. IH $\sum_{i=1}^k i = k(k + 1)/2$. Then $\sum_{i=1}^{k+1} i = k(k + 1)/2 + (k + 1) = (k + 1)(k + 2)/2$. $\square$

1.3.2 Strong Induction

Assume all $P(j)$, $n_0 \leq j \leq k$, to get $P(k+1)$.

Theorem 1.2 (Every $n \geq 2$ has a prime divisor). For $n \geq 2$, n divisible by a prime.

Proof. Base 2 prime. Fix $k \geq 2$, assume every $2 \leq j \leq k$ has a prime divisor. If $k+1$ prime done; else $k+1 = a \cdot b$, $2 \leq a, b \leq k$, prime divisor of a divides $k+1$. $\square$

Theorem 1.3 (Fibonacci growth). $F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2} \Rightarrow F_n \geq 2^{n/2-1}$ for $n \geq 1$.

Proof. Bases $n = 1, 2$ check. IH for $1 \leq j \leq k$, $k \geq 2$: $F_{k+1} = F_k + F_{k-1} \geq 2^{k/2-1} + 2^{(k-1)/2-1} \geq 2^{(k+1)/2-1}$ since $\sqrt{2} + 1 > 2$. $\square$

1.3.3 Structural Induction

For recursively defined sets (lists, trees), prove base structures and that constructors preserve P .

Example 1.6 (Binary trees). Tree is Leaf or Node(L, x, R). To prove “leaves = internal + 1” for full trees, check leaf base and show constructor preserves invariant.

1.4 What Is an Algorithm?

Definition 1.2 (Algorithm, informal). A finite, unambiguous sequence of elementary steps that, given any admissible input, terminates and produces the required output.

Knuth’s five criteria: finiteness, definiteness, input, output, effectiveness—plus termination on every admissible input. A procedure that may loop forever is a computational method, not an algorithm. An algorithm is distinct from a program: the idea vs. its realisation in Python/Java/C with the same asymptotics.

1.4.1 Pseudocode Conventions

Language-agnostic, one-indexed arrays (CLRS style), // comments, $\leftarrow$, $=$, $\neq$, $<$, $\leq$, **if-then-else**, **while**, **for**, **return**. Count primitive ops (§1.5) not wall-clock time.

Example 1.7 (Euclid — oldest non-trivial algorithm). EUCLID(a, b) // assume $a \geq b \geq 0$

```

1 while  $b \neq 0$  do
2    $r \leftarrow a \bmod b$ 
3    $a \leftarrow b$ 
4    $b \leftarrow r$ 
5 return  $a$ 
```

We prove in §1.6 this returns $\gcd(a, b)$ and in Ch. 2 it costs $O(\log \min(a, b))$ divisions.

1.5 The Random Access Machine (RAM) Model

To compare algorithms fairly we need a shared cost model: the Random Access Machine, an idealised von Neumann computer. Concretely: memory $M[0 \dots m-1]$ of words ($\Theta(\log n)$ bits) with $O(1)$ random access;

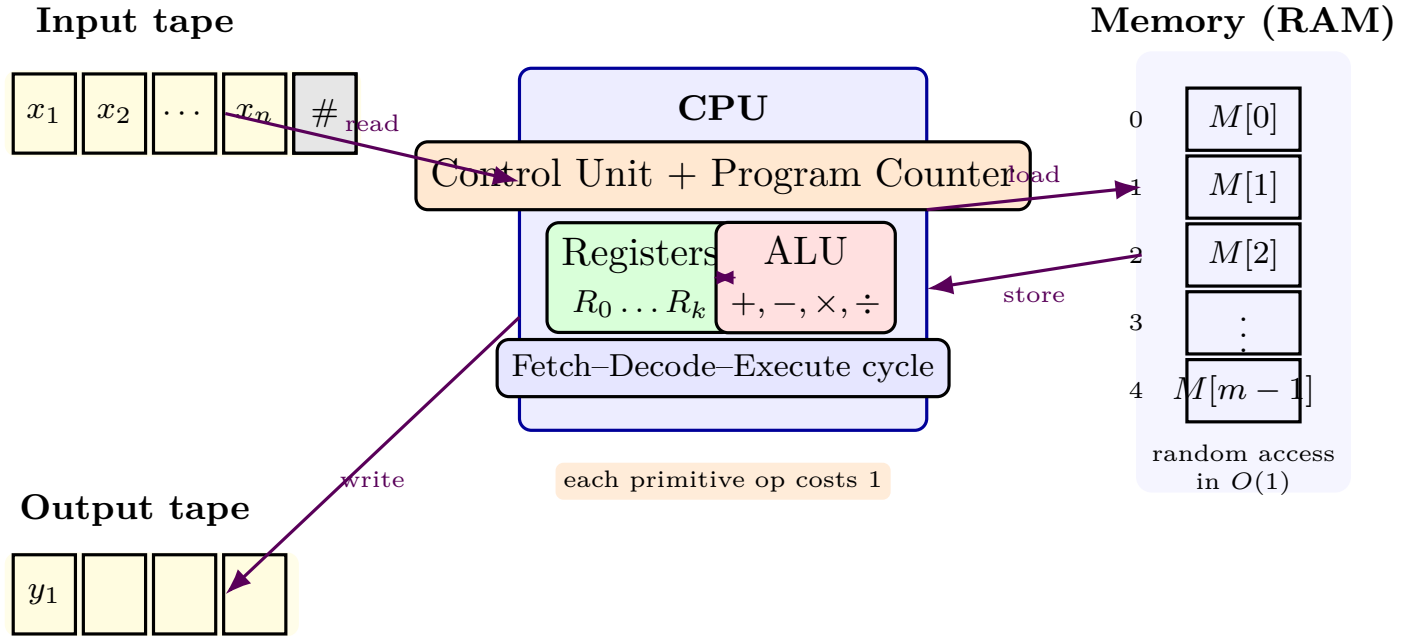


Figure 1.2: The RAM. CPU fetches via program counter, operates on registers/ALU, accesses any $M[i]$ in unit time.

CPU registers + ALU + program counter; instruction set LOAD, STORE, ADD, SUB, MUL, DIV, MOD, COMPARE, JUMP, JZ, READ, WRITE each cost 1 (uniform cost); program stored separately.

1.5.1 Cost Model and Word-RAM Refinement

Running time = number of primitive steps. Input size in words (or bits when bit-complexity matters). Uniform cost suffices while numbers fit in a word ($w = \Theta(\log n)$ bits). For huge integers, arithmetic on b -bit values costs $\Theta(b/w)$; we flag when bit-complexity matters. Why this model? It abstracts caches/pipelines yet predicts correctly: $O(n \log n)$ beats $O(n^2)$ past a modest n , letting us reason before implementing.

Example 1.8 (Cost of linear search). `LINEARSEARCH( $A, n, key$ )`

```

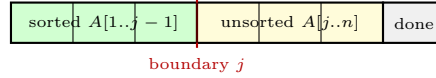
1  for  $i \leftarrow 1$  to  $n$  do
2      if  $A[i] = key$  then return  $i$ 
3  return NIL
```

Each iteration does $O(1)$ RAM ops (array access + compare + branch); time $\propto$ iterations = $O(n)$ worst-case.

1.6 Correctness: Loop Invariants and Termination

An algorithm is useless if wrong or non-terminating. *Correctness* = partial correctness (if it stops, output meets spec) + termination. For loops: invariant for the former, variant for the latter.

Remark 1.2 (Recipe — words before formal). When you write an invariant, answer: What is done? What remains? Where is the boundary? State it in words first (“first $j - 1$ slots already sorted”), then formalise.



Invariant = “left of barrier sorted & is permutation of original prefix”

Definition 1.3 (Loop invariant). A predicate I on program state is a *loop invariant* for loop guard G if:

- (LI1) **Initialisation:** I holds before first iteration.
- (LI2) **Maintenance:** if I holds before an iteration and G true, then I holds after.
- (LI3) **Termination:** $I \wedge \neg G$ implies desired postcondition.

Think of I as the inductive hypothesis travelling around the loop—its proof *is* induction on iteration count. A *variant* v (integer, strictly decreasing, bounded below, usually ≥ 0) guarantees termination.

Example 1.9 (Insertion sort — full correctness). INSERTIONSORT($A[1 \dots n]$)

```

1  for  $j \leftarrow 2$  to  $n$  do
2       $key \leftarrow A[j]$ 
3       $i \leftarrow j - 1$ 
4      while  $i > 0$  and  $A[i] > key$  do
5           $A[i + 1] \leftarrow A[i]$ 
6           $i \leftarrow i - 1$ 
7       $A[i + 1] \leftarrow key$ 

```

Spec: $A[1..n]$ is a permutation of input and non-decreasing. Outer invariant (words first): *first $j - 1$ slots are the original $A_0[1..j - 1]$ in sorted order*. Formally $A[1..j - 1] = \text{sorted}(A_0[1..j - 1])$. Initialisation $j = 2$: $A[1..1]$ trivially sorted. Maintenance: inner loop shifts sorted block until position for $key = A_0[j]$ found, yielding sorted $A[1..j]$. Termination $j = n + 1$: $A[1..n]$ sorted permutation. Inner invariant: $A[i + 2..j]$ holds elements $> key$ shifted by one; $A[1..i]$ sorted. Variant i decreases to 0; outer variant $n - j$. Hence total correctness.

Example 1.10 (Euclid terminates). Variant $v = b$: $(a, b) \mapsto (b, a \bmod b)$ with $0 \leq a \bmod b < b$ so v strictly decreases ≥ 0 . Invariant $\text{gcd}(a, b)$ unchanged ($\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$) gives postcondition $a = \text{gcd}(a_0, b_0)$.

1.7 Chapter Notes

The RAM model is due to Cook and Reckhow (1973) and Aho, Hopcroft, Ullman; our version follows CLRS Ch. 2; word-RAM is standard (Demaine). Loop invariants are Hoare logic in miniature (Hoare, 1969); full method in SC3270. For discrete math, Rosen and MH1812 notes remain references.

1.8 Exercises

E1.1 **Warm-up: logic and quantifiers.** Formalise “Every SC2001 student who passed MH1812 can follow an induction proof” using predicates, then write its negation without $\neg$ in front of a quantifier. Why is

negation *not* “No student can”?

E1.2 Induction — weak vs. strong. Prove $\sum_{i=1}^n i^2 = n(n+1)(2n+1)/6$ by induction. Then prove every $n \geq 12$ is $4a + 5b$ ($a, b \geq 0$). Which needed strong induction? (Hint: bases 12, 13, 14, 15.)

E1.3 RAM cost model.

```
SUMARRAY( $A[1 \dots n]$ )
1   $s \leftarrow 0$ 
2  for  $i \leftarrow 1$  to  $n$  do  $s \leftarrow s + A[i]$ 
3  return  $s$ 
```

(a) Count RAM ops/line and total time. (b) If each $A[i]$ is $b \gg \log n$ bits, how does word-RAM $w = \Theta(\log n)$ change it? (c) $O(n)$ or $O(nb/w)$?

E1.4 Loop invariant — find the bug. Invariant for LINEARSEARCH: “At start of iteration i , $key \notin A[1..i-1]$.” Prove correctness if used correctly. Give a buggy variant where invariant holds but algorithm incorrect on termination; which of (LI1)–(LI3) fails?

E1.5 Euclid’s invariant in full. Show (a) $\gcd(a, b) = \gcd(b, a \bmod b)$ for $b > 0$, (b) invariant holds/when $b = 0$ postcondition, (c) $O(\log b)$ iterations via $a \bmod b < a/2$ when $b \leq a$.

Chapter 2

Asymptotic Analysis

“We should forget about small efficiencies, say about 97% of the time.” — Knuth. Asymptotic notation lets us forget the 97% and compare the shape of growth itself.

Prerequisites: Ch. 1 (RAM model, induction, sums/logs). This chapter formalises the $O(\cdot)$ language used in every later analysis.

Learning Outcomes

After this chapter you will be able to:

- (L1) state the five asymptotic notations with quantifiers and negate them correctly;
- (L2) apply the limit method to classify functions;
- (L3) order the hierarchy $\log n \prec n \prec n \log n \prec n^2 \prec 2^n$ and justify it;
- (L4) distinguish best-, worst-, and average-case analysis;
- (L5) write a recurrence for a recursive algorithm’s running time (bridge to Ch. 3).

2.1 Why Asymptotics?

RAM time is $T(n) = \sum(\text{cost}_i \times \text{count}_i(n))$, an unwieldy exact formula. Constants depend on compiler and hardware; what survives across machines is *rate of growth*. Asymptotic notation suppresses constants and lower-order terms and compares $T(n)$ to a clean reference $g(n)$ for large n only.

Two programs solve the same task: A takes $f_A(n) = 3n^2 + 5n$ steps, B takes $f_B(n) = 100n$ steps (constant factors from hardware). For small n , B may be slower; for large n , shape wins and B crushes A regardless of the $100\times$ constant. “Large n ” means $\exists n_0 \forall n \geq n_0$.

We consider $f, g : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ eventually non-negative; “after n_0 ” is the formal handle for “for large n .”

n	$3n^2 + 5n$	$100n$	winner
10	350	1 000	A ($350 < 1\,000$)
100	30 500	10 000	B
1 000	3 005 000	100 000	B by $30\times$
10 000	300 050 000	1 000 000	B by $300\times$

Table 2.1: Shape overtakes constants: quadratic $3n^2 + 5n$ beats $100n$ only below the crossover $n_0 \approx 33$; after that the linear algorithm wins forever.

2.2 The Five Notations

Picture first. $f = O(g)$ means “ f stays under a scaled copy of g after some cutoff n_0 ” (one suitable c suffices). Ω is the mirror (stays above), Θ is sandwiched between two scalings, while o/ω require the sandwich for *every* $c > 0$ — the band can be made arbitrarily tight (dashed narrowing bands in Fig. 2.1). The formal $\exists c \exists n_0 \forall n \geq n_0$ below is just that picture in quantifiers.

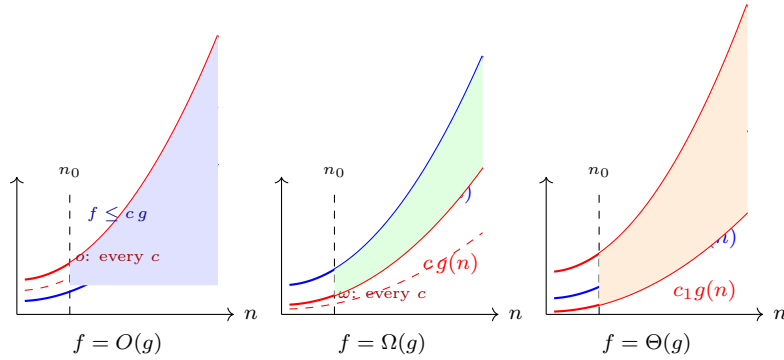


Figure 2.1: O , Ω , Θ as eventually-sandwiching bands (color fills). Dashed red curves show o/ω : the sandwich must hold for *every* $c > 0$, so the band can be narrowed arbitrarily. Only $n \geq n_0$ matters.

Definition 2.1 (O — asymptotic upper bound). $f(n) = O(g(n))$ iff $\exists c > 0 \exists n_0 \in \mathbb{N} \forall n \geq n_0 : 0 \leq f(n) \leq cg(n)$.

Definition 2.2 (Ω — asymptotic lower bound). $f(n) = \Omega(g(n))$ iff $\exists c > 0 \exists n_0 \forall n \geq n_0 : 0 \leq cg(n) \leq f(n)$.

Definition 2.3 (Θ — tight bound). $f(n) = \Theta(g(n))$ iff $\exists c_1, c_2 > 0 \exists n_0 \forall n \geq n_0 : 0 \leq c_1g(n) \leq f(n) \leq c_2g(n)$. Equivalently $f = O(g)$ and $f = \Omega(g)$.

Definition 2.4 (o — strict upper bound). $f(n) = o(g(n))$ iff $\forall c > 0 \exists n_0 \forall n \geq n_0 : 0 \leq f(n) < cg(n)$. That is, $f/g \rightarrow 0$.

Definition 2.5 (ω — strict lower bound). $f(n) = \omega(g(n))$ iff $\forall c > 0 \exists n_0 \forall n \geq n_0 : 0 \leq cg(n) < f(n)$. That is, $f/g \rightarrow \infty$.

Remark 2.1 (Quantifier order matters). O has $\exists c \exists n_0 \forall n \geq n_0$; o has $\forall c \exists n_0 \forall n \geq n_0$. Negations swap quantifiers: $f \neq O(g)$ iff $\forall c > 0 \forall n_0 \exists n \geq n_0 : f(n) > cg(n)$. The abuse $f = O(g)$ means $f \in O(g)$ as a set.

2.3 The Limit Method

Intuition (no calculus needed). If the ratio $f(n)/g(n)$ settles near a constant, f and g grow at the same rate (Θ); if it tends to 0, f is negligible (o); if it blows up, f dominates (ω). Numbers first: for $f(n) = 3n^2 + 5n$, $g(n) = n^2$,

n	$f(n)/g(n) = (3n^2 + 5n)/n^2$	$\rightarrow$
10	3.50	
100	3.05	3
1 000	3.005	

The ratio $\rightarrow 3 \in (0, \infty)$, so we expect $f = \Theta(g)$. The theorem below makes this precise; l'Hôpital is just an optional calculus shortcut for the limit, not the definition.

Theorem 2.1 (Limit test). Let $L = \lim_{n \rightarrow \infty} f(n)/g(n)$ (in $\mathbb{R}_{\geq 0} \cup \{\infty\}$), assuming $g(n) > 0$ eventually.

- (a) If $0 < L < \infty$ then $f = \Theta(g)$.
- (b) If $L = 0$ then $f = o(g)$ (hence $f = O(g)$).
- (c) If $L = \infty$ then $f = \omega(g)$ (hence $f = \Omega(g)$).

If L does not exist (oscillation), the test is inconclusive and we return to definitions.

Example 2.1. $3n^2 + 5n + 20 = \Theta(n^2)$ because $(3n^2 + 5n + 20)/n^2 \rightarrow 3 \in (0, \infty)$. Similarly $\log(n!) = \Theta(n \log n)$ since $n! \leq n^n$ and $n! \geq (n/2)^{n/2}$, or via Stirling; we prove it rigorously in Ch. 3.

Example 2.2 (Limits need care). $f(n) = n^{1+\sin n}$ oscillates between n^0 and n^2 ; $f(n)/n$ has no limit and f is neither $O(n)$ nor $\Omega(n^2)$. Always check existence.

Optional shortcut: l'Hôpital for ∞/∞ , and $\log_a n = \Theta(\log_b n)$ for any $a, b > 1$ (change of base is a constant factor). Hence we write simply $\Theta(\log n)$.

2.4 Growth Hierarchy

Theorem 2.2 (Hierarchy). For any $\epsilon > 0$ and $c > 1$:

$$1 \prec \log n \prec n^\epsilon \prec n \prec n \log n \prec n^{1+\epsilon} \prec n^2 \prec 2^n \prec n! \prec n^n,$$

where $a(n) \prec b(n)$ means $a(n) = o(b(n))$. In particular $\log n = o(n^\epsilon)$, $n^k = o(c^n)$ for any fixed k , and $c^n = o(n!)$.

Sketch. Each claim is a limit: e.g. $\log n/n^\epsilon \rightarrow 0$ by l'Hôpital, and $n^k/c^n \rightarrow 0$ because exponentials dominate polynomials (apply l'Hôpital k times). The rest is transitivity of o . $\square$

Fig. 2.2 shows the separation. Consequence: an $O(n \log n)$ sorting algorithm will beat an $O(n^2)$ one for large n no matter the constants — the motivation for Ch. 4.

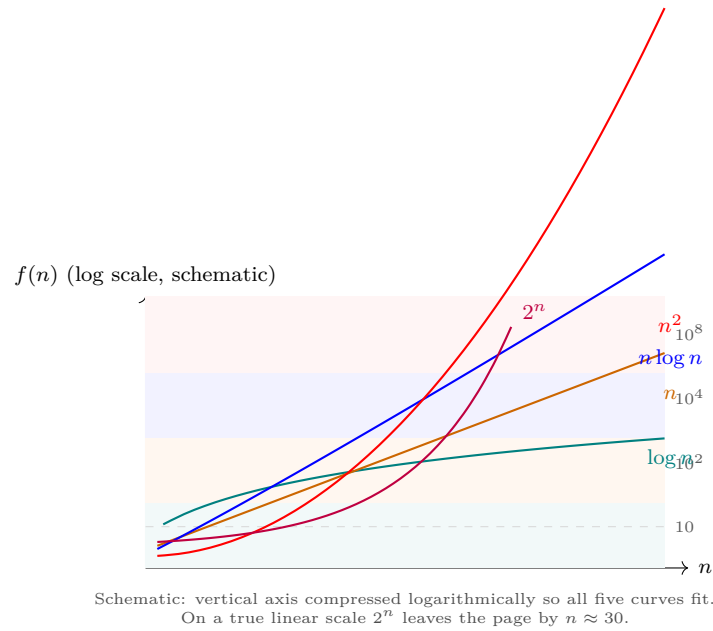


Figure 2.2: Growth hierarchy on a logarithmically compressed vertical axis. Steeper eventually dominates: $\log n \ll n \ll n \log n \ll n^2 \ll 2^n$.

2.5 Best, Worst, and Average Case

Micro-primer: expectation. Fix N inputs of size n and pick one uniformly at random (each with prob. $1/N$). The *average* running time is the ordinary average $\mathbb{E}[T] = \frac{1}{N} \sum_{i=1}^N T(I_i)$. For an event, the *indicator* $X = \mathbf{1}[\text{event}]$ is 1 if it happens, 0 otherwise, so $\mathbb{E}[X] = \Pr[\text{event}]$ and $\mathbb{E}[\sum X_j] = \sum \mathbb{E}[X_j]$ (linearity).

For input size n , let $T(I)$ be RAM steps on instance I with $|I| = n$:

- **Worst case:** $T_{\text{worst}}(n) = \max_{|I|=n} T(I)$ — the guarantee.
- **Best case:** $T_{\text{best}}(n) = \min_{|I|=n} T(I)$ — rarely used alone (overly optimistic).
- **Average case:** $T_{\text{avg}}(n) = \mathbb{E}_{I \sim D_n}[T(I)]$ under a distribution D_n over size- n inputs (often uniform). Requires specifying D_n .

When we say “LINEARSEARCH is $O(n)$ ” we mean $T_{\text{worst}}(n) = O(n)$; its best case is $\Theta(1)$ and its average case (uniform, key present) is $\Theta(n)$. INSERTIONSORT is $\Theta(n)$ best case (already sorted, inner loop never runs) but $\Theta(n^2)$ worst case.

Average-case analysis needs the distribution D_n stated explicitly. Randomised algorithms (Ch. 8) instead randomise the *algorithm* and bound $\mathbb{E}[T(I)]$ for every I .

2.6 Bridge to Recurrences

Recursive algorithms do not have closed-form $T(n)$ directly; they satisfy *recurrences*.

Example 2.3 (Merge sort preview). MERGESORT on n elements splits into two halves, recurses, then merges

in $\Theta(n)$:

$$T(n) = 2T(n/2) + \Theta(n), \quad T(1) = \Theta(1).$$

Solving gives $T(n) = \Theta(n \log n)$ — the canonical divide-and-conquer recurrence solved in Ch. 3.

In general a recurrence has form $T(n) = aT(n/b) + f(n)$ or $T(n) = T(n-1) + f(n)$ with base $T(n_0) = c_0$. Ch. 3 develops three systematic solvers: substitution (guess-and-verify by induction), recursion trees (summing level costs), and the Master Theorem for $aT(n/b) + f(n)$. Asymptotics from this chapter is the language in which those solutions are stated.

2.7 Chapter Notes

CLRS Ch. 3 is the standard reference for $O/\Theta/o/\omega$; Knuth (1976) introduced O, Ω, Θ notation into CS. The limit method is an application of real analysis — Rudin Ch. 3. Hierarchy facts follow from $\lim_{n \rightarrow \infty} (\log n)/n^\epsilon = 0$, proved via l'Hôpital. Average-case analysis connects to probability (MH1813); we revisit it for QuickSort and hashing.

2.8 Exercises

- E2.1 Definitions and negations.** Let $f(n) = 5n^2 + 3n$ and $g(n) = n^2$. (a) Prove $f = O(g)$ by exhibiting c, n_0 . (b) Prove $f = \Omega(g)$. Conclude $f = \Theta(g)$. (c) Write the formal negation of “ $h(n) = o(g(n))$ ” with quantifiers and use it to show $n^2 \neq o(n^2)$.
- E2.2 Limit method vs. oscillation.** Classify each pair using the limit method when applicable; otherwise return to definitions: (a) $f(n) = n \log n$, $g(n) = n^{1.5}$; (b) $f(n) = 2^n$, $g(n) = 3^n$; (c) $f(n) = n^{1+\sin n}$, $g(n) = n$. Determine which of $O, \Omega, \Theta, o, \omega$ hold (list all that apply).
- E2.3 Cases and distributions.** Consider LINEARSEARCH for a key in $A[1 \dots n]$ (distinct elements, key present uniformly at random position). (a) Give $T_{\text{best}}(n)$, $T_{\text{worst}}(n)$ with Θ bounds. (b) Compute $T_{\text{avg}}(n)$ exactly as a sum and give its Θ class. (c) Suppose the key is absent with probability $1/2$ and otherwise uniform among positions. How does $T_{\text{avg}}(n)$ change?
- E2.4 Setting up recurrences.** Write recurrences (with base cases) for: (a) RECURSIVEMAX that finds the maximum by comparing $A[n]$ with the max of $A[1 \dots n-1]$; (b) binary search on a sorted array of size n (worst case); (c) Karatsuba-style: $T(n) = 3T(n/2) + \Theta(n)$ — do not solve, just explain in one sentence what 3 , $n/2$, and $\Theta(n)$ represent. Identify which cases from (a)–(c) fit the Master form $aT(n/b) + f(n)$.

Chapter 3

Recurrences

Recurrences describe the running time of recursive algorithms. Solving them—turning a recursive definition into a closed asymptotic bound—is the central analytic step in divide-and-conquer analysis. This chapter develops three complementary tools, highlights their limits, and applies them to MergeSort and Karatsuba multiplication.

3.1 From Algorithms to Recurrences

A typical divide-and-conquer algorithm on input size n divides into a subproblems of size n/b , then spends $f(n)$ time dividing and combining.

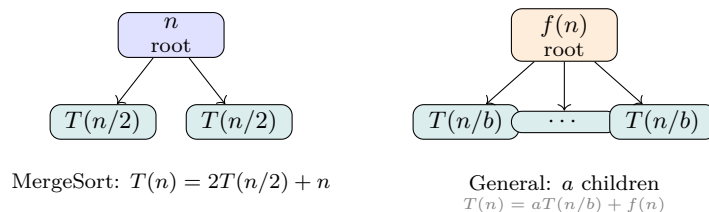


Figure 3.1: One-level expansion: peel off the root cost $f(n)$ to reveal a subproblems of size n/b (MergeSort instance left, general form right). Repeating this expansion yields the full recursion tree.

Its worst-case time satisfies

$$T(n) = aT(n/b) + f(n), \quad T(1) = \Theta(1), \quad (3.1)$$

with $a \geq 1$, $b > 1$ constants and f asymptotically positive. We assume n is an exact power of b ; floors and ceilings change bounds by at most a constant factor. *Running examples.* MergeSort: $T(n) = 2T(n/2) + \Theta(n)$. Karatsuba (Sec. 3.4.1): $T(n) = 3T(n/2) + \Theta(n)$. Binary search: $T(n) = T(n/2) + \Theta(1)$. Strassen: $T(n) = 7T(n/2) + \Theta(n^2)$. Our goal is a Θ -bound for $T(n)$ —methods in order of increasing automation and decreasing generality.

Remark 3.1 (Floors, ceilings, and base cases). Replacing $T(n/b)$ by $T(\lfloor n/b \rfloor)$ or $T(\lceil n/b \rceil)$ does not change the asymptotic answer; sandwich between exact-power variants. Base cases $T(1) = \Theta(1)$ vs. $T(n) = O(1)$ for $n \leq n_0$ are interchangeable up to constants.

3.2 The Substitution Method

Guess a bound, then verify by induction. The method is fully general but requires insight; recursion trees (next section) are the usual source of guesses.

Bridge—tree suggests, substitution proves. The tree in Fig. 3.2 will suggest MergeSort costs cn per level $\times \log n$ levels $= cn \log n$. Here we *verify* that guess rigorously; the next example shows the $cn - b$ trick when a naive guess leaves a $+1$ residue.

Example 3.1 (MergeSort upper bound). Claim $T(n) = 2T(n/2) + n$ with $T(1) = 1$ satisfies $T(n) = O(n \log n)$. Guess $T(n) \leq cn \log n$ for $c > 0$ and $n \geq 2$. Assume it holds for $n/2$:

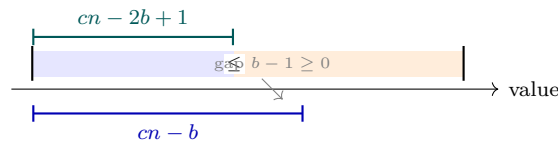
$$T(n) \leq 2c \frac{n}{2} \log \frac{n}{2} + n = cn \log n - cn + n \leq cn \log n$$

provided $c \geq 1$. Choosing c large enough to handle base cases completes the induction. Symmetric lower bound gives $T(n) = \Theta(n \log n)$.

Example 3.2 (Strengthening the hypothesis). To prove $T(n) = T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + 1$ is $O(n)$, naive $T(n) \leq cn$ fails: $T(n) \leq c\lfloor n/2 \rfloor + c\lceil n/2 \rceil + 1 = cn + 1$. Strengthen to $T(n) \leq cn - b$ for $c, b > 0$:

$$T(n) \leq c\lfloor n/2 \rfloor - b + c\lceil n/2 \rceil - b + 1 = cn - 2b + 1 \leq cn - b$$

once $b \geq 1$. Subtracting a low-order term absorbs the additive constant.



The $cn - b$ trick shifts the guess down by b so the $+1$ residue is swallowed: $cn - 2b + 1 \leq cn - b \iff b \geq 1$.

When to use it: only method that proves matching upper and lower bounds with full rigour, including floors.
Pitfall: guess must be tight; with floors use $n/2 - 1 < \lfloor n/2 \rfloor \leq n/2$.

Change of variables. For $T(n) = 2T(\sqrt{n}) + \log n$, set $m = \log n$, $S(m) = T(2^m)$ to get $S(m) = 2S(m/2) + m$ (MergeSort in m), so $S(m) = \Theta(m \log m)$ and $T(n) = \Theta(\log n \cdot \log \log n)$.

3.3 The Recursion-Tree Method

Expand the recurrence into a tree, sum level by level, then verify the guess by substitution. The tree is discovery; substitution remains the proof. For (3.1), level 0 has cost $f(n)$, level 1 has a nodes each costing $f(n/b)$, level i has a^i nodes each costing $f(n/b^i)$. Depth $L = \log_b n$,

$$T(n) = \sum_{i=0}^{L-1} a^i f(n/b^i) + \Theta(n^{\log_b a}), \quad (3.2)$$

where last term is leaf cost ($a^L = n^{\log_b a}$ leaves each $\Theta(1)$). Bounding this sum is the entire problem. Fig. 3.2 illustrates the expansion.

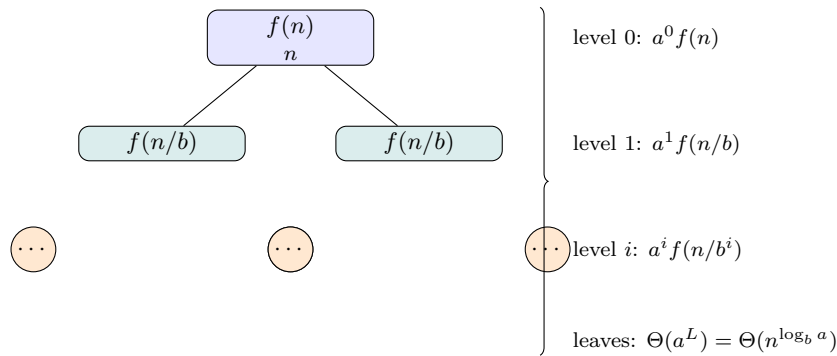


Figure 3.2: Recursion tree for $T(n) = aT(n/b) + f(n)$. Total cost is sum over levels plus leaves; $L = \log_b n$.

Example 3.3 (MergeSort via tree). $T(n) = 2T(n/2) + cn$. Level i has 2^i nodes of cost $cn/2^i = cn$, so each level costs cn . With $\log_2 n$ levels, $T(n) = cn \log_2 n + \Theta(n) = \Theta(n \log n)$.

Example 3.4 (Karatsuba preview). $T(n) = 3T(n/2) + cn$. Level i costs $3^i \cdot c(n/2^i) = cn(3/2)^i$ —geometric, ratio $3/2 > 1$, dominated by last term:

$$\sum_{i=0}^{L-1} cn(3/2)^i = cn \frac{(3/2)^L - 1}{3/2 - 1} = \Theta(n \cdot n^{\log_2 3 - 1}) = \Theta(n^{\log_2 3}).$$

The Master theorem automates this.

Remark 3.2. Trees handle recurrences outside the Master form, e.g. $T(n) = T(n/3) + T(2n/3) + \Theta(n)$, where longest path is $\log_{3/2} n$ and careful bounding still yields $\Theta(n \log n)$ —settled directly by Akra–Bazzi. In general, if $a^i f(n/b^i)$ grows (resp. shrinks) by a constant factor each level, the sum is dominated by the last (resp. first) term.

Recurrence with non-constant work. $T(n) = 2T(n/2) + n \log n$: level i has 2^i nodes each $(n/2^i) \log(n/2^i)$, so level cost $n(\log n - i)$. Summing gives $n \sum_{i=0}^{L-1} (\log n - i) = \Theta(n \log^2 n)$, matching Master Case 2 with $k = 1$.

3.4 The Master Theorem

The Master theorem classifies (3.1) when f is polynomially comparable to $n^{\log_b a}$. It replaces tree summation with a case check.

Walk-through (Fig. 3.3). Plot $a^i f(n/b^i)$ versus i . **Shrinking** (blue, Case 1): level cost falls geometrically, last level $\approx n^{c^*}$ dominates, so $T(n) = \Theta(n^{c^*})$ —e.g. Karatsuba $3T(n/2) + n$. **Flat** (orange, Case 2): every level contributes $\Theta(n^{c^*})$, $\log_b n$ levels give $\Theta(n^{c^*} \log n)$ —e.g. MergeSort $2T(n/2) + n$. **Increasing** (red, Case 3): cost grows geometrically, root $f(n)$ dominates, so $T(n) = \Theta(f(n))$ provided regularity $af(n/b) \leq cf(n)$ forces geometric decay upward.

Theorem 3.1 (Master theorem). Let $a \geq 1$, $b > 1$, f asymptotically positive, and $T(n) = aT(n/b) + f(n)$. Let $c_\star = \log_b a$.

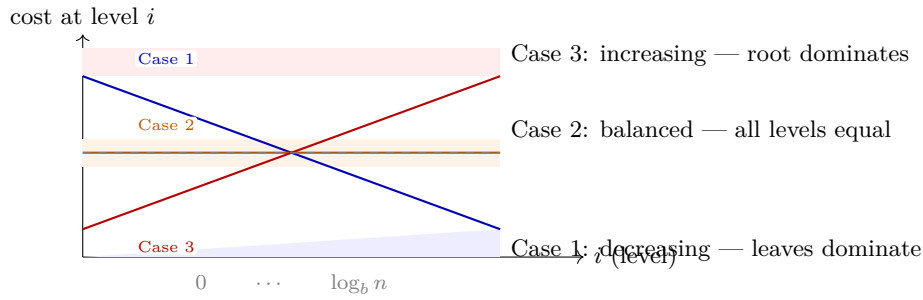


Figure 3.3: Master geometry: per-level cost $a^i f(n/b^i)$ across levels. The dominant contribution determines the case—read the picture before the theorem.

Case 1: If $f(n) = O(n^{c_\star - \varepsilon})$ for some $\varepsilon > 0$, then $T(n) = \Theta(n^{c_\star})$.

Case 2: If $f(n) = \Theta(n^{c_\star} \log^k n)$ for some $k \geq 0$, then $T(n) = \Theta(n^{c_\star} \log^{k+1} n)$. In particular $f(n) = \Theta(n^{c_\star})$ gives $\Theta(n^{c_\star} \log n)$.

Case 3: If $f(n) = \Omega(n^{c_\star + \varepsilon})$ for some $\varepsilon > 0$ and the *regularity* condition $af(n/b) \leq cf(n)$ holds for some $c < 1$ and large n , then $T(n) = \Theta(f(n))$.

Proof sketch. Write $g(n) = f(n)/n^{c_\star}$ so $a^i f(n/b^i) = n^{c_\star} g(n/b^i)$. Then (3.2) = $n^{c_\star} \sum_i g(n/b^i) + \Theta(n^{c_\star})$. Case 1: $g(n) = O(n^{-\varepsilon})$, sum is convergent geometric $\Rightarrow \Theta(n^{c_\star})$. Case 2: $g(n) = \Theta(\log^k n)$, sum over $L = \Theta(\log n)$ terms $\Rightarrow \Theta(\log^{k+1} n)$. Case 3: regularity gives $a^i f(n/b^i) \leq c^i f(n)$, geometric decay toward leaves, sum = $\Theta(f(n))$.

Why regularity is needed. Without it Case 3 is false: contrived f with $f(n) = \Omega(n^{c_\star + \varepsilon})$ can dip sharply at infinitely many n/b^i , breaking geometric decay. For polynomials it is automatic—check $af(n/b)/f(n) = a/b^k < 1$ when $f(n) = n^k$, $k > c_\star$.

Gaps. The theorem does *not* cover every f . Need $\varepsilon > 0$ polynomial gap: $f(n) = n^{c_\star} \log \log n$ and $f(n) = n^{c_\star} / \log n$ fall strictly between Cases 1 and 2.

Gap box ($n = 1024$). Take $c_\star = 2$ so $n^{c_\star} = 1\,048\,576$. Then $n^{c_\star} / \log_2 n = 1\,048\,576/10 = 104\,858$ while $n^{c_\star} \log \log n \approx 1\,048\,576 \times 3.32 \approx 3.48\text{M}$. Neither is $O(n^{2-\varepsilon})$ nor $\Omega(n^{2+\varepsilon})$ nor $\Theta(n^2)$ for any fixed $\varepsilon > 0$; the ratio $f(n)/n^{c_\star} = 1/\log n$ (or $\log \log n$) tends to 0 (or ∞) slower than any $n^{\pm\varepsilon} = 1024^{\pm\varepsilon}$. Hence no Master case applies—use tree/Akra–Bazzi. The true cost is *between* $n^{c_\star}$ and $n^{c_\star} \log n$.

How to apply it. (1) Compute $c_\star = \log_b a$. (2) Compare $f(n)$ to $n^{c_\star}$: polynomially smaller / Theta-equivalent (up to $\log^k n$) / polynomially larger with regularity? (3) If none holds, Master is silent—use tree or Akra–Bazzi.

3.4.1 Worked Applications

MergeSort. $a = 2, b = 2, c_\star = 1, f(n) = \Theta(n) = \Theta(n^{c_\star})$: Case 2 $\Rightarrow \Theta(n \log n)$.

Karatsuba. n -digit $X = a \cdot 10^m + b, Y = c \cdot 10^m + d, m = n/2$. Three products suffice: $T(n) = 3T(n/2) + \Theta(n)$. Here $c_\star = \log_2 3 \approx 1.585$, $f(n) = O(n^{c_\star - 0.58})$: Case 1 $\Rightarrow \Theta(n^{\log_2 3})$, beating naive $\Theta(n^2)$.

Strassen. $T(n) = 7T(n/2) + \Theta(n^2)$: $c_\star = \log_2 7 \approx 2.807 > 2$, Case 1 $\Rightarrow \Theta(n^{\log_2 7})$.

Binary search. $T(n) = T(n/2) + \Theta(1)$: $c_\star = 0$, $f(n) = \Theta(1)$: Case 2 $\Rightarrow \Theta(\log n)$.

Quick check. $T(n) = 9T(n/3) + \Theta(n^2)$: $c_\star = 2$, $f = \Theta(n^{c_\star})$ so Case 2 $\Rightarrow \Theta(n^2 \log n)$; $f = n^3 \Rightarrow$ Case 3; $f = n \Rightarrow$ Case 1.

Relation of $f(n)$ to $n^{c_\star}$	Case	$T(n)$	Intuition
$f(n) = O(n^{c_\star - \varepsilon})$	1	$\Theta(n^{c_\star})$	Leaves dominate
$f(n) = \Theta(n^{c_\star} \log^k n)$	2	$\Theta(n^{c_\star} \log^{k+1} n)$	Balanced levels
$f(n) = \Omega(n^{c_\star + \varepsilon})$, regular	3	$\Theta(f(n))$	Root dominates
Gaps: $n^{c_\star}/\log n$, $n^{c_\star} \log \log n$	—	—	Use tree / Akra–Bazzi

Table 3.1: Master theorem at a glance ($c_\star = \log_b a$). Memory aid—still check ε and regularity.

3.5 Beyond Equal Subproblems: Akra–Bazzi

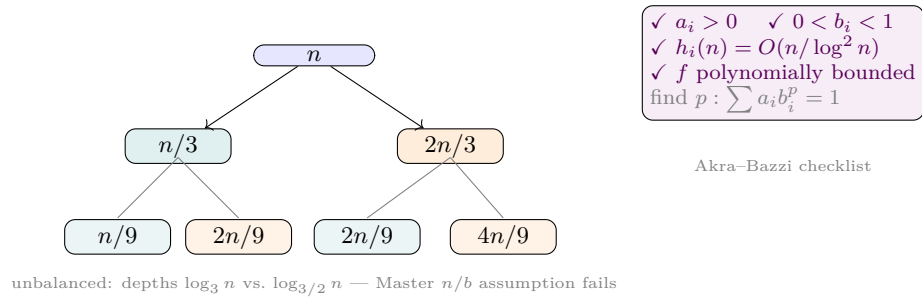


Figure 3.4: Lopsided recursion for $T(n) = T(n/3) + T(2n/3) + \Theta(n)$ and Akra–Bazzi applicability checklist.

When subproblem sizes differ or f is not polynomially separated, Master does not apply. Akra–Bazzi handles

$$T(n) = \sum_{i=1}^k a_i T(b_i n + h_i(n)) + f(n), \quad 0 < b_i < 1, \quad a_i > 0,$$

with $h_i(n) = O(n/\log^2 n)$. Let p solve $\sum_i a_i b_i^p = 1$. Then

$$T(n) = \Theta\left(n^p \left(1 + \int_1^n \frac{f(u)}{u^{p+1}} du\right)\right).$$

For Master form $aT(n/b) + f(n)$ this recovers all three cases. For $T(n) = T(n/3) + T(2n/3) + \Theta(n)$ we get $p = 1$ and $\int_1^n u^{-1} du = \log n$, so $T(n) = \Theta(n \log n)$. Check: $T(n) = 2T(n/4) + \Theta(\sqrt{n})$ has $2(1/4)^p = 1 \Rightarrow p = 1/2$, $\int u^{1/2}/u^{3/2} du = \log n$, so $\Theta(\sqrt{n} \log n)$, matching Master Case 2.

Remark 3.3 (Using Akra–Bazzi in SC2001). You may quote it without reproving, but verify hypotheses: $a_i > 0$, $0 < b_i < 1$, $h_i(n) = O(n/\log^2 n)$, f polynomially bounded. For $T(n) = T(n/2) + T(n/4) + \Theta(n)$, $(1/2)^p + (1/4)^p = 1 \Rightarrow p \approx 0.694$; integral is $\Theta(n)$ -dominated $\Rightarrow T(n) = \Theta(n)$.

3.6 Chapter Notes and Exercises

Substitution builds proof discipline; trees build intuition; Master automates the common case. Table 3.1 summarises. When subproblems are unequal, floors are messy, or f oscillates, fall back to trees or Akra–Bazzi. Habit: conjecture with tree, confirm dominant case with Master when applicable, verify by substitution.

Exercise 3.1 (Substitution practice). Solve $T(n) = 4T(n/2) + n$ with $T(1) = 1$. Guess $T(n) = O(n^2)$ and prove by substitution. Then prove $\Omega(n^2)$ to conclude $\Theta(n^2)$. Which Master case?

Exercise 3.2 (Recursion tree). Use a tree to conjecture bounds for (a) $T(n) = 2T(n/2) + n^2$ and (b) $T(n) = T(n/2) + T(n/4) + \Theta(n)$. Verify one by substitution. For which does Master apply?

Exercise 3.3 (Master theorem gaps). For each state which Master case applies (or why none) and give bound: (a) $T(n) = 2T(n/2) + n \log n$, (b) $T(n) = 2T(n/2) + n/\log n$, (c) $T(n) = 4T(n/2) + n^2 \log n$, (d) $T(n) = 2T(n/2) + \Theta(n^{1.5})$.

Exercise 3.4 (Karatsuba vs. naive). Karatsuba $T_K(n) = 3T_K(n/2) + \Theta(n)$ vs. naive $T_N(n) = 4T_N(n/2) + \Theta(n)$. Compute both bounds and find smallest $n = 2^k$ where $n^{\log_2 3} < n^2/10$.

Chapter 4

Divide and Conquer

“Divide et impera.” — The oldest strategy in warfare is also one of the most powerful in algorithm design: break a hard problem into smaller copies of itself, solve them recursively, and combine.

Prerequisites: Recurrences and asymptotics (Ch. 2), induction (Ch. 1), basic probability — see Micro-primer §4.2 for indicators and linearity used in QuickSort.

Learning Outcomes

After this chapter you will be able to:

- (L1) state the D&C paradigm and write recurrences for D&C algorithms;
- (L2) prove correctness and $\Theta(n \log n)$ complexity of MergeSort;
- (L3) analyse deterministic vs. randomised QuickSort (worst case vs. expected $O(n \log n)$ via indicator variables);
- (L4) prove the strip sparsity lemma for the closest-pair problem;
- (L5) derive Strassen’s 7-multiplication recurrence and its $O(n^{\log_2 7})$ bound.

4.1 The Divide-and-Conquer Paradigm

A D&C algorithm for input size n does three steps:

1. **Divide** the instance into $a \geq 1$ subinstances each of size $\lceil n/b \rceil$ or $\lfloor n/b \rfloor$ ($b > 1$).
2. **Conquer** by solving subinstances recursively (base case $n \leq n_0$ solved directly).
3. **Combine** sub-solutions into a solution for the original instance at cost $f(n)$.

Hence

$$T(n) = aT(n/b) + f(n), \quad T(1) = \Theta(1). \quad (4.1)$$

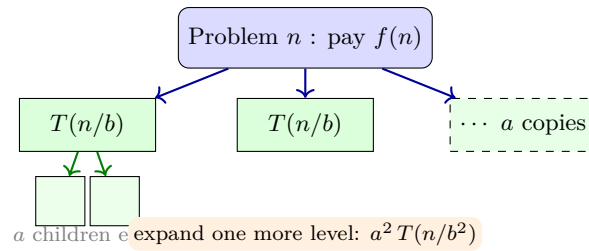


Figure 4.1: Two-level D&C tree: root combine cost $f(n)$ plus a subproblems $T(n/b)$. Summing levels gives $T(n) = aT(n/b) + f(n)$, $T(1) = \Theta(1)$.

The Master Theorem (Ch. 3, Thm. 3.1) classifies (4.1): if $f(n) = O(n^{\log_b a - \epsilon})$ then $T(n) = \Theta(n^{\log_b a})$; if $f(n) = \Theta(n^{\log_b a})$ then $T(n) = \Theta(n^{\log_b a} \log n)$; if $f(n) = \Omega(n^{\log_b a + \epsilon})$ and regular then $T(n) = \Theta(f(n))$. D&C wins when the combine cost is cheap relative to shrinkage; the next sections make this concrete.

4.2 MergeSort

MergeSort($A[p..r]$): if $p < r$, let $q = \lfloor (p+r)/2 \rfloor$, recursively sort $A[p..q]$ and $A[q+1..r]$, then **Merge** the two sorted halves in $\Theta(n)$ time using an auxiliary array.

Theorem 4.1 (Correctness of MergeSort). **MergeSort** returns a permutation of the input sorted in nondecreasing order.

Proof. Induction on $n = r - p + 1$. Base $n = 1$ trivially sorted. Assume holds for all sizes $< n$. Both halves are sorted by IH. **Merge** repeatedly takes the smaller leading element, so output is sorted and contains exactly $A[p..r]$. By induction the claim holds. $\square$

Merge scans each element once, so $f(n) = \Theta(n)$. Thus $T(n) = 2T(n/2) + \Theta(n)$, $T(1) = \Theta(1)$. Here $a = 2$, $b = 2$, $\log_b a = 1$, $f(n) = \Theta(n) = \Theta(n^{\log_b a})$, so Master case 2 gives $T(n) = \Theta(n \log n)$. This is asymptotically optimal for comparison sorting (Ch. 8). MergeSort guarantees $\Theta(n \log n)$ worst-case but needs $\Theta(n)$ extra space.

Micro-primer: Indicator Variables and Linearity of Expectation

For an event E the *indicator* $X = \mathbf{1}[E]$ is 1 if E occurs else 0; then $\mathbb{E}[X] = \Pr[E]$ because $\mathbb{E}[X] = 1 \cdot \Pr[E] + 0 \cdot \Pr[\bar{E}]$. *Linearity*: for any X_i (dependent or not), $\mathbb{E}[\sum_i X_i] = \sum_i \mathbb{E}[X_i]$; expectation of a sum is the sum of expectations, by definition $\sum_x x \Pr[X = x]$. *Example (2 coins)*: toss fair coins with $X_1 = \mathbf{1}[\text{coin 1 is H}]$, $X_2 = \mathbf{1}[\text{coin 2 is H}]$; $\mathbb{E}[X_i] = 1/2$, $\mathbb{E}[X_1 + X_2] = 1$ even though X_1, X_2 are independent, and $\mathbb{E}[2X_1] = 1$ by scaling. For QuickSort we write total comparisons $X = \sum_{i < j} X_{ij}$ with each $X_{ij} \in \{0, 1\}$ and use $\mathbb{E}[X] = \sum_{i < j} \Pr[X_{ij} = 1]$ without needing independence.

4.3 QuickSort

Partition(A, p, r) picks pivot $x = A[r]$, maintains i with $A[p..i] \leq x$ and $A[i+1..j-1] > x$, scanning j from p to $r-1$ and swapping when $A[j] \leq x$. Finally swap $A[i+1]$ with $A[r]$; return $q = i+1$. Cost is $\Theta(n)$.

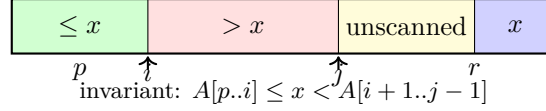


Figure 4.2: Lomuto partition invariant.

QuickSort(A, p, r): if $p < r$, $q \leftarrow \text{Partition}(A, p, r)$, recurse on $A[p..q-1]$ and $A[q+1..r]$.

Deterministic worst case. If pivot is always smallest or largest (e.g., sorted input with $A[r]$ as pivot), partitions are 0 and $n-1$: $T(n) = T(n-1) + \Theta(n) = \Theta(\sum_{k=1}^n k) = \Theta(n^2)$.

Balanced partitions ($n/4$ – $3n/4$) give $T(n) \leq 2T(3n/4) + \Theta(n) = \Theta(n \log n)$, but the adversary can force $\Theta(n^2)$.

Randomised QuickSort. Choose pivot uniformly at random from $A[p..r]$ (or randomly permute first). Let $z_1 < \dots < z_n$ be sorted values and $X_{ij} = 1$ if z_i and z_j are ever compared (one is chosen as pivot while both in same call), else 0. Total comparisons $X = \sum_{i < j} X_{ij}$.

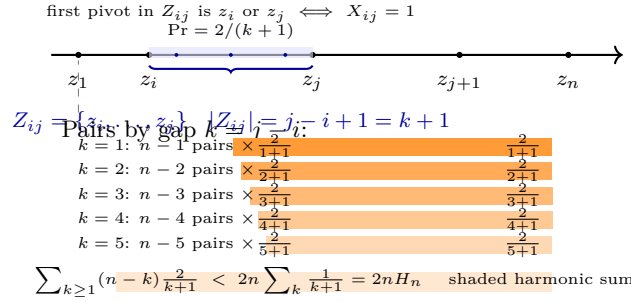


Figure 4.3: Indicator view: interval Z_{ij} on sorted line (blue) and triangle of $\frac{2}{k+1}$ terms by gap k ; shading = harmonic sum H_n .

For $i < j$, z_i and z_j are compared iff among Z_{ij} the first pivot chosen is z_i or z_j . All $|Z_{ij}| = j - i + 1$ are equally likely first, so $\Pr[X_{ij} = 1] = 2/(j - i + 1)$ and $\mathbb{E}[X_{ij}] = 2/(j - i + 1)$. By linearity (Micro-primer),

$$\mathbb{E}[X] = \sum_{i < j} \frac{2}{j - i + 1} = \sum_{i=1}^n \sum_{k=1}^{n-i} \frac{2}{k+1} < 2 \sum_i H_n = O(n \log n),$$

more precisely $\mathbb{E}[X] = 2(n+1)H_n - 4n \sim 2n \ln n$. Each comparison is $O(1)$, so $\mathbb{E}[T(n)] = O(n \log n)$. Randomisation removes the adversary: *every* input has expected $O(n \log n)$.

4.4 Closest Pair of Points

Given $P = \{p_1, \dots, p_n\} \subset \mathbb{R}^2$, find $\min_{i \neq j} \|p_i - p_j\|_2$. Brute force is $\Theta(n^2)$. D&C: sort by x , split by median $x = m$ into P_L, P_R , recursively find δ_L, δ_R , set $\delta = \min(\delta_L, \delta_R)$, then check pairs straddling the line within vertical strip $|x - m| < \delta$.

Lemma 4.1 (Strip sparsity). Any $\delta \times 2\delta$ rectangle in the strip contains at most 8 points of P (hence at most 7 successors need be checked; tighter packing gives 6).

Proof. Partition the rectangle into 8 squares of side $\delta/2$ (2 columns $\times$ 4 rows). Diagonal is $\delta/\sqrt{2} < \delta$. If two

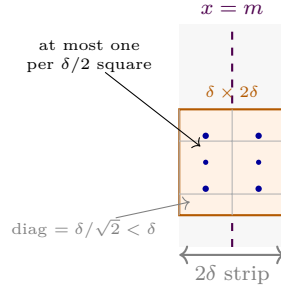


Figure 4.4: Strip sparsity: any $\delta \times 2\delta$ rectangle in the 2δ strip splits into 8 squares of side $\delta/2$; diagonal $\delta/\sqrt{2} < \delta$, so each square holds at most one point — at most 7 successors need be checked.

points lay in the same square their distance would be $< \delta$, contradicting that distances within P_L and P_R are $\geq \delta$. So at most one per square. $\square$

Consequently, for each point in the strip sorted by y , only the next 7 points need be checked. The strip scan is $O(n)$ after $O(n \log n)$ presorting (or $O(n)$ with presorted arrays passed down). Recurrence: $T(n) = 2T(n/2) + O(n) \Rightarrow T(n) = O(n \log n)$. This beats $\Theta(n^2)$ and is optimal in the algebraic decision-tree model. In practice the y -order is maintained by passing the y -sorted array through recursion.

Remark 4.1. The closest-pair recurrence is identical in form to MergeSort's; the difficulty is entirely in the combine step. The sparsity lemma is what keeps $f(n) = O(n)$.

4.5 Strassen's Matrix Multiplication

Naive $n \times n$ multiplication costs $\Theta(n^3)$. Partition $A = [A_{11} \ A_{12}; A_{21} \ A_{22}]$, $B = [B_{11} \ B_{12}; B_{21} \ B_{22}]$, $C = AB$ similarly, each block $n/2 \times n/2$. Standard block multiplication needs 8 half-size products. Strassen (1969) uses 7, like Karatsuba's trick: ab normally needs 4 products $(a_1b_1, a_1b_0, a_0b_1, a_0b_0)$ but $(a_1 + a_0)(b_1 + b_0) - a_1b_1 - a_0b_0$ recovers the cross terms with 3; Strassen does the same for 2×2 blocks with 7 products M_k (each a sum/difference of blocks).

$$M_1 = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$M_5 = (A_{11} + A_{12})B_{22}$$

$$M_2 = (A_{21} + A_{22})B_{11}$$

$$M_6 = (A_{21} - A_{11})(B_{11} + B_{12})$$

$$M_3 = A_{11}(B_{12} - B_{22})$$

$$M_7 = (A_{12} - A_{22})(B_{21} + B_{22})$$

$$M_4 = A_{22}(B_{21} - B_{11})$$

Then $C_{11} = M_1 + M_4 - M_5 + M_7$, $C_{12} = M_3 + M_5$, $C_{21} = M_2 + M_4$, $C_{22} = M_1 - M_2 + M_3 + M_6$. Each M_k is one $n/2 \times n/2$ product; the 18 additions cost $\Theta(n^2)$.

Recurrence: $T(n) = 7T(n/2) + \Theta(n^2)$, $T(1) = \Theta(1)$. Here $\log_b a = \log_2 7 \approx 2.807$ and $f(n) = O(n^2) = O(n^{\log_2 7 - \epsilon})$, so Master case 1 gives $T(n) = \Theta(n^{\log_2 7}) = O(n^{2.81})$, asymptotically faster than $\Theta(n^3)$. Applied recursively (padding to power of two if needed), it is practical for large n despite larger constants.

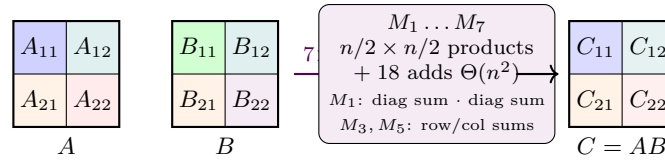


Figure 4.5: Strassen: one $n \times n$ product reduced to seven $n/2 \times n/2$ products M_k (annotated: which block sums/differences) plus $O(n^2)$ additions.

4.6 Exercises

- 4.1 MergeSort invariants.** Write the loop invariant for `Merge( $L, R$ )` that merges two sorted arrays of total length n . Prove initialization, maintenance, and termination, and show comparisons $\leq n - 1$.
- 4.2 QuickSort indicators.** Let X_{ij} be as in §4.3. (a) Justify $\Pr[X_{ij} = 1] = 2/(j - i + 1)$ by symmetry among Z_{ij} . (b) Compute $\mathbb{E}[X]$ exactly as $2(n + 1)H_n - 4n$ and deduce $\mathbb{E}[X] \leq 2n \ln n + O(n)$. What if pivots are $A[r]$ on a random permutation input?
- 4.3 Closest-pair tightness.** (a) Prove the strip lemma with 8 squares side $\delta/2$ implies at most 7 successors need be checked. (b) Give 8 points achieving 6 in a $\delta \times 2\delta$ rectangle with all pairwise distances $\geq \delta$, showing 7 cannot be reduced to 3 without finer argument.
- 4.4 Strassen in practice.** (a) Verify algebraically that the seven M_k yield the correct C_{ij} . (b) Show for $n = 2^k$ the count satisfies $T(n) = 7^k + 6 \cdot \sum_{i=0}^{k-1} 7^i 4^{k-1-i}$ and confirm $T(n) = \Theta(n^{\log_2 7})$. At what n_0 switch to naive if Strassen's constant is c_s vs. c_0 ?

Chapter 5

Greedy Algorithms

“Make the best local choice and never look back.” — The greedy motto. Sometimes it yields optimal solutions; often it does not. The craft lies in knowing which and proving it.

Prerequisites: Asymptotic notation (Ch. 2), exchange arguments and induction (Ch. 1), priority queues and Union-Find (Ch. 3), basic graph terminology (Ch. 6 preview).

Learning Outcomes

After this chapter you will be able to:

- (L1) describe the greedy paradigm and state the greedy-choice and optimal-substructure conditions;
- (L2) apply the exchange-argument template to prove correctness of a greedy algorithm;
- (L3) prove optimality of earliest-finish-time interval scheduling;
- (L4) construct Huffman codes and prove their optimality;
- (L5) state and apply the cut property to justify Kruskal’s and Prim’s MST algorithms;
- (L6) exhibit inputs where the greedy heuristic fails (coin change).

5.1 The Greedy Paradigm and the Exchange Argument

A *greedy* algorithm builds a solution incrementally, choosing at each step the locally best option according to some criterion, never reconsidering earlier choices.

Definition 5.1 (Greedy choice & optimal substructure). An optimisation problem has the *greedy-choice property* if a globally optimal solution can be assembled by making a locally optimal first choice and then solving the remaining subproblem optimally. It has *optimal substructure* if an optimal solution contains optimal solutions to subproblems.

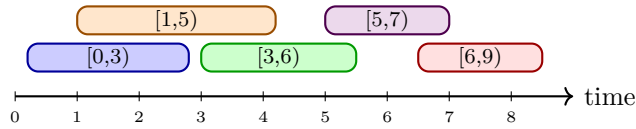
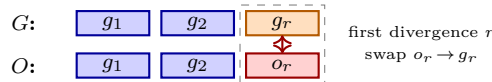


Figure 5.1: Intervals on a timeline. Earliest finish (blue) leaves the most room — scan left-to-right, no back-tracking.

Greedy is not universally correct; it requires both properties. Proofs of correctness almost always use an *exchange argument*:



Make O look more like G without hurting it — the core of every exchange proof.

Exchange template. Let G be the greedy solution and O an optimal solution that agrees with G for as long as possible (first k choices). Show the $(k+1)$ st greedy choice can be exchanged into O without worsening its value, obtaining O' that agrees with G one step further and remains optimal. By induction G is optimal.

Two recurring lemmas support the exchange: (i) *greedy stays ahead* (after k steps the greedy partial solution is at least as good as any competitor's), and (ii) *cut/exchange* (swapping in the greedy element does not hurt feasibility or cost).

5.2 Interval Scheduling: Earliest Finish Time

Given n intervals $[s_i, f_i)$ (half-open), find a maximum-cardinality subset of pairwise compatible (non-overlapping) intervals.

```

1 EarliestFinish(intervals):
2   sort by finish time f_1 <= f_2 <= ... <= f_n
3   S = empty; last = -inf
4   for i = 1..n:
5       if s_i >= last: S.add(i); last = f_i
6   return S
    
```

Theorem 5.1 (Optimality of earliest finish). EarliestFinish returns a maximum-size compatible set.

Exchange argument. Let $G = \langle g_1, \dots, g_k \rangle$ be the greedy set in order of finish time and $O = \langle o_1, \dots, o_m \rangle$ an optimal set ordered the same way, chosen to maximise the longest common prefix with G . Let r be the first index where they differ (or $r = k + 1$ if G is a prefix of O). If $r > k$ then $|G| \leq |O|$ and, since G is maximal, $|G| = |O|$. Otherwise $f_{g_r} \leq f_{o_r}$ (greedy picks the earliest-finishing compatible interval) and $s_{o_{r+1}} \geq f_{o_r} \geq f_{g_r}$, so $O' = (O \setminus \{o_r\}) \cup \{g_r\}$ is feasible and $|O'| = |O|$. O' is optimal and agrees with G one step further, contradicting maximality of r . Hence no such r exists and $|G| = |O|$. $\square$

The argument shows *greedy stays ahead*: $f_{g_i} \leq f_{o_i}$ for all $i \leq \min(k, m)$. This invariant is the core of many greedy proofs—compare the k th greedy choice directly to the k th choice of any competitor.

Example 5.1 (Small instance). Intervals $[1, 4), [3, 5), [0, 6), [5, 7), [8, 9)$ sorted by finish are $[1, 4), [3, 5), [0, 6), [5, 7), [8, 9)$. Greedy picks $[1, 4), [5, 7), [8, 9)$ (size 3), optimal; $[1, 4) \rightarrow [0, 6)$ cannot improve.

Running time is $O(n \log n)$ for sorting plus $O(n)$ scanning. Variants (earliest start time, shortest interval, fewest conflicts) all fail—counterexamples are left as exercises.

5.3 Huffman Coding

Given alphabet C with frequencies $\text{freq}(c) > 0$, find a binary *prefix code* (no codeword is a prefix of another) minimising weighted path length $B(T) = \sum_c \text{freq}(c) \cdot d_T(c)$, where $d_T(c)$ is depth in the coding tree T .

Huffman's greedy rule: repeatedly merge the two lowest-frequency nodes.

```

1 Huffman(C, freq):
2   Q = min-heap of leaves by freq
3   for i = 1 to |C|-1:
4     x = Q.extractMin(); y = Q.extractMin()
5     z = internal node with children x,y, freq(z)=freq(x)+freq(y)
6     Q.insert(z)
7   return Q.extractMin() // root

```

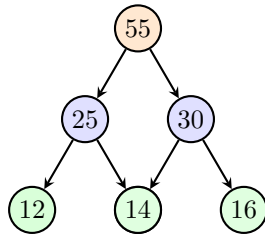
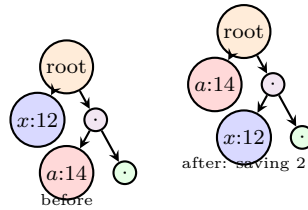


Figure 5.2: Huffman tree sketch: internal labels are summed frequencies; leaves carry character frequencies (here 12, 13, 14, 16). The two smallest (12, 13) are siblings at maximum depth—the greedy choice.

Lemma 5.1 (Sibling property). There exists an optimal prefix tree in which the two least-frequent characters are siblings at maximum depth.

Numeric swap demo (before Δ). Rare x (freq 12, depth 2) vs. frequent a (freq 14, depth 3):

	freq	depth	$freq \cdot depth$			freq	depth	$freq \cdot depth$
x	12	2	24	$\rightarrow$	x	12	3	36
a	14	3	42		a	14	2	28
before			66		after			64



Saving = $(14 - 12)(3 - 2) = 2 \geq 0$; in general $\Delta = (\text{freq}(a) - \text{freq}(x))(d(a) - d(x))$.

Exchange. Take an optimal tree T and let x, y be the two least-frequent characters. Let a, b be siblings at maximum depth in T . If $\{x, y\} = \{a, b\}$ we are done. Otherwise swap x with a (and y with b if needed). Since $\text{freq}(x) \leq \text{freq}(a)$ and $d(x) \leq d(a)$ cannot both be reversed, each swap does not increase $B(T)$: $\Delta = (\text{freq}(a) - \text{freq}(x))(d(a) - d(x)) \geq 0$ cost is not raised. The resulting tree is optimal with x, y as deepest siblings. $\square$

Theorem 5.2 (Huffman optimality). Huffman produces an optimal prefix code.

Induction + exchange. Induction on $|C| = n$. Base $n = 2$ trivial. For $n > 1$, let x, y be the two smallest frequencies merged into z with $\text{freq}(z) = \text{freq}(x) + \text{freq}(y)$. Let T' be the optimal tree for $C' = (C \setminus \{x, y\}) \cup \{z\}$ given by induction (Huffman on C'). Expand z into x, y to get T ; then $B(T) = B(T') + \text{freq}(x) + \text{freq}(y)$. If some tree T^* for C had $B(T^*) < B(T)$, transform T^* by the Lemma so x, y are siblings, contract them to z to get T_0^* for C' with $B(T_0^*) = B(T^*) - \text{freq}(x) - \text{freq}(y) < B(T')$, contradicting optimality of T' . $\square$

Complexity is $O(n \log n)$ with a heap. Decoding is unambiguous precisely because the code is prefix-free—the tree path is followed until a leaf is reached. Optimal merging and the structure of prefix codes both hinge on the same exchange idea.

Remark 5.1. Kraft's inequality $\sum_c 2^{-d_T(c)} \leq 1$ characterises feasible prefix-code lengths. Huffman achieves minimum $\sum \text{freq}(c) d_T(c)$, i.e. minimum redundancy above entropy $H = -\sum p_c \log p_c$.

5.4 Minimum Spanning Trees: The Cut Property

Let $G = (V, E)$ be connected, undirected, with distinct edge weights $w(e)$ (ties broken arbitrarily). A *spanning tree* minimises total weight among all spanning trees if it is an MST.

Picture a cut first: partition V into S and $V \setminus S$ (dashed bubble). Which crossing edge must every MST take? The lightest one.

Definition 5.2 (Cut property). For any cut $(S, V \setminus S)$, the minimum-weight edge crossing the cut belongs to every MST.

Exchange. Let $e = (u, v)$ be the lightest edge crossing $(S, V \setminus S)$ and suppose some MST T omits e . Adding e to T creates a unique cycle C , which must cross the cut in another edge $f \neq e$. Since $w(e) < w(f)$, $T' = (T \setminus \{f\}) \cup \{e\}$ is a spanning tree lighter than T —contradiction. Hence $e \in T$. $\square$

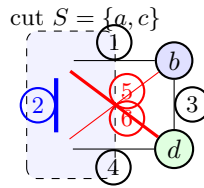


Figure 5.3: Cut $(S, V \setminus S)$, $S = \{a, c\}$. Crossing edges weigh 1, 3, 5, 6; the lightest (weight 1) must be in every MST (cut property below).

Both classic algorithms are greedy instantiations of this property.

```

1 Kruskal(V, E, w):
2   sort E by w ascending; make Union-Find on V
3   T = empty
4   for e=(u,v) in sorted E:
5     if Find(u) != Find(v): T.add(e); Union(u,v) % Find=already connected? Union=merge
6   return T

```

Each edge added by Kruskal is the lightest crossing the cut $(C, V \setminus C)$ where C is its component before the union.

```

1 Prim(V, E, w, r):
2   S={r}; T=empty; PQ = edges leaving S keyed by w % key = cheapest edge leaving S
3   while S != V:
4     e=(u,v)=PQ.extractMin() with u in S, v not in S
5     T.add(e); S.add(v); insert edges from v to V\S into PQ
6   return T

```

Each edge added by Prim is exactly the minimum edge of cut $(S, V \setminus S)$.

Correctness of both follows immediately from the cut property: every edge they add is forced into every MST, so the final tree is contained in all MSTs and hence is one itself. Kruskal runs in $O(|E| \log |E|)$ (sorting dominates); Prim with a binary heap is $O(|E| \log |V|)$, $O(|E| + |V| \log |V|)$ with a Fibonacci heap. The cycle property (heaviest edge on any cycle is never in any MST) is dual and also proved by exchange.

5.5 When Greedy Fails: Coin Change

Given denominations $D = \{d_1 < \dots < d_k\}$ and amount A , minimise number of coins. The greedy rule—repeatedly take the largest $d_i \leq$ remaining amount—is optimal for canonical systems (e.g. US coins $\{1, 5, 10, 25\}$) but not in general.

Example 5.2 (Counterexample). $D = \{1, 3, 4\}$, $A = 6$. Greedy picks $4 + 1 + 1$ (3 coins) while $3 + 3$ uses 2 coins. For $D = \{1, 10, 25\}$, $A = 30$: greedy gives $25 + 1 + 1 + 1 + 1 + 1$ (6 coins) vs. $10 + 10 + 10$ (3 coins).

The DP table for $D = \{1, 3, 4\}$, $A = 6$ illustrates the fix:

x	0	1	2	3	4	5	6
$dp[x]$	0	1	2	1	1	2	2
choice	—	1	1	3	4	1+4	3+3

Greedy is myopic: taking 4 at $x = 6$ forces two 1s, while dp considers all predecessors $x - d_i$. More generally, no fixed greedy order works for all D . The problem is solved optimally by dynamic programming in $O(kA)$:

$$dp[0] = 0, \quad dp[x] = \min_{d_i \leq x} \{1 + dp[x - d_i]\}.$$

The failure illustrates the missing greedy-choice property: a locally optimal coin can block a globally better combination.

5.6 Exercises

- E1. **Interval variants that fail.** Give counterexamples showing that (a) earliest start time, (b) shortest interval first, and (c) fewest conflicts first are *not* optimal for interval scheduling.
- E2. **Huffman trace.** For frequencies $\{5, 9, 12, 13, 16, 45\}$, simulate Huffman step-by-step, draw the final tree, compute $B(T)$, and verify the average code length against the entropy lower bound.
- E3. **Coin-change classification.** Prove that greedy is optimal for $D = \{1, 5, 10, 25\}$ (US coins without the half-dollar). Then find the smallest A where greedy fails for $D = \{1, 3, 4\}$ and prove dp above is correct.
- E4. **Cut/cycle duality.** Prove the *cycle property*: if e is the maximum-weight edge on some cycle, no MST contains e . Deduce that the MST is unique when all edge weights are distinct.
- E5. **Scheduling with deadlines.** Each job has duration t_i and deadline d_i . The greedy order by earliest deadline minimises maximum lateness $L_{\max} = \max_i (f_i - d_i)$ where f_i is completion time. Prove optimality via an exchange argument (adjacent swap).

Chapter Notes

Huffman (1952), Kruskal/Prim (1956–57): matroid optimisation; interval scheduling $\rightarrow$ weighted (Ch. 7 DP). U.S. coins canonical—Pearson (2005). See matroids/greedoids.

Chapter 6

Dynamic Programming

Dynamic programming (DP) solves optimisation problems by combining solutions to overlapping subproblems. Unlike divide-and-conquer, which solves independent subproblems, DP reuses work—trading space for time and often reducing exponential search to polynomial time.

6.1 The DP Paradigm

Two properties characterise problems amenable to DP.

6.1.1 Optimal Substructure

A problem has *optimal substructure* if an optimal solution contains optimal solutions to its subproblems. Formally, let S be an optimal solution to instance I and let I' be a sub-instance induced by S ; then the restriction of S to I' is optimal for I' .

Not every optimisation problem qualifies. Longest *simple* path in a general graph lacks optimal substructure—the subpath of a longest simple path need not be longest between its endpoints, because reusing vertices may be forbidden globally but not locally. DP is inapplicable there.

6.1.2 Overlapping Subproblems

A recursive formulation has *overlapping subproblems* when the recursion tree revisits the same sub-instances many times. Naive recursion then does exponential work. DP stores each distinct subproblem result once.

Property	Divide & Conquer	Dynamic Programming
Subproblems	Independent / disjoint	Overlapping
Recomputation	None	Extensive without memo
Typical fix	–	Memoisation or tabulation

Table 6.1: DP versus divide-and-conquer.

6.1.3 Memoisation versus Tabulation

Both techniques achieve the same asymptotic bound; they differ in control flow.

- **Memoisation (top-down):** Write the natural recursion and cache results in a dictionary or array on first computation. Only subproblems reachable from the initial call are solved. Recursion overhead and stack depth remain.
- **Tabulation (bottom-up):** Enumerate subproblems in order of increasing size and fill a table iteratively. No recursion; often better cache locality and easier complexity analysis.

```

1  # memoised Fibonacci
2  memo = {0:0, 1:1}
3  def fib(n):
4      if n not in memo:
5          memo[n] = fib(n-1) + fib(n-2)
6      return memo[n]
7
8  # tabulated Fibonacci
9  def fib_tab(n):
10     dp = [0]*(n+1); dp[1]=1
11     for i in range(2, n+1):
12         dp[i] = dp[i-1] + dp[i-2]
13     return dp[n]
```

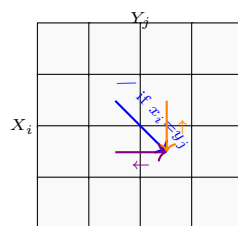
Tabulation is preferred in this course because it makes the dependency structure explicit, which we exploit to reconstruct solutions.

6.2 Longest Common Subsequence

Given $X = x_1 \dots x_m$ and $Y = y_1 \dots y_n$, a *subsequence* deletes zero or more characters. The LCS problem asks for a longest sequence that is a subsequence of both X and Y .

6.2.1 Optimal Substructure and Recurrence

Picture a grid with X down the rows and Y across the columns. The *longest zigzag* from $(0,0)$ to (m,n) —step $\diagup$ on a match, otherwise $\uparrow$ or $\leftarrow$ —collects the most matching characters; its length *is* the LCS.



empty c table: each cell depends on 3 neighbours; longest zigzag = LCS

Let $c[i][j]$ be the LCS length of $X_i = x_1 \dots x_i$ and $Y_j = y_1 \dots y_j$, with $c[0][j] = c[i][0] = 0$.

$$c[i][j] = \begin{cases} c[i-1][j-1] + 1 & \text{if } x_i = y_j, \\ \max\{c[i-1][j], c[i][j-1]\} & \text{if } x_i \neq y_j. \end{cases}$$

If $x_i = y_j$, any LCS must end with that character and extend an LCS of X_{i-1}, Y_{j-1} . If $x_i \neq y_j$, an LCS discards either x_i or y_j . Overlapping subproblems are immediate: $c[i][j]$ depends on three neighbours, so naive recursion re-evaluates the same cells exponentially many times.

6.2.2 Tabulation and Walkthrough

Filling row by row gives $O(mn)$ time and $O(mn)$ space. We also store a direction $b[i][j] \in \{\swarrow, \uparrow, \leftarrow\}$ to reconstruct an LCS by backtracking from $c[m][n]$.

Take $X = \langle A, B, C \rangle$, $Y = \langle A, C, B \rangle$ ($m = n = 3$):

	0	A	C	B
0	0	0	0	0
A	0	1	1	1
B	0	1	1	2
C	0	1	2	2

c table; blue cells = matches, arrows = backtrack for LCS $\langle A, B \rangle$

Filling order: $c[1][1] = 1$ ($A=A$, diagonal $+1$), $c[1][2] = 1$ ($\max(0, 1)$), $c[1][3] = 1$, $c[2][3] = 2$ ($B=B$). Backtracking from $c[3][3] = 2$ follows the arrows above to recover $\langle A, B \rangle$ (equivalently $\langle A, C \rangle$ via a different tie-break).

Reconstruction is $O(m+n)$: start at (m, n) ; on $\swarrow$ output x_i and move to $(i-1, j-1)$; on $\uparrow$ move to $(i-1, j)$; on $\leftarrow$ move to $(i, j-1)$.

6.3 0/1 Knapsack

Given n items with weights w_i , values v_i , and capacity W , choose a subset maximising total value without exceeding W . Each item is taken at most once.

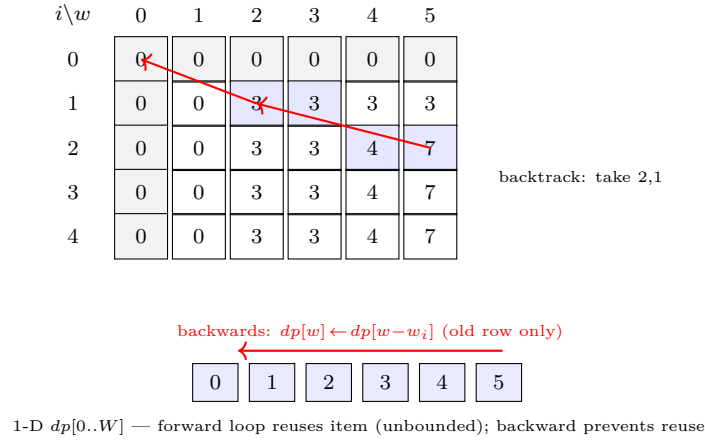
Let $K[i][w]$ be the optimum value using items $1 \dots i$ with capacity w , where $K[0][w] = K[i][0] = 0$:

$$K[i][w] = \begin{cases} K[i-1][w] & \text{if } w_i > w, \\ \max\{K[i-1][w], K[i-1][w-w_i] + v_i\} & \text{otherwise.} \end{cases}$$

Either item i is excluded (value $K[i-1][w]$) or included (value $K[i-1][w-w_i] + v_i$). Time $O(nW)$, space $O(nW)$ —pseudo-polynomial in W .

6.3.1 Table and Backtracking

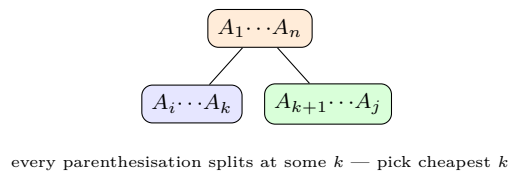
Example: $W = 5$, items (w, v) : 1 : (2, 3), 2 : (3, 4), 3 : (4, 5), 4 : (5, 6).



Optimum $K[4][5] = 7$ (items 1+2, weight 5). Backtrack: if $K[i][w] \neq K[i-1][w]$, item i was taken; move to $w-w_i$, else move to $i-1$. Space can be reduced to $O(W)$ with a 1-D array iterated backwards, but backtracking then needs the full table or stored decisions.

6.4 Matrix Chain Multiplication

Given matrices $A_1 \dots A_n$ where A_i is $p_{i-1} \times p_i$, the cost of multiplying $A_i \dots A_j$ depends on parenthesisation. Multiplying $p \times q$ by $q \times r$ costs pqr scalar multiplications.



Let $m[i][j]$ be the minimum cost to compute $A_i \dots A_j$, with $m[i][i] = 0$:

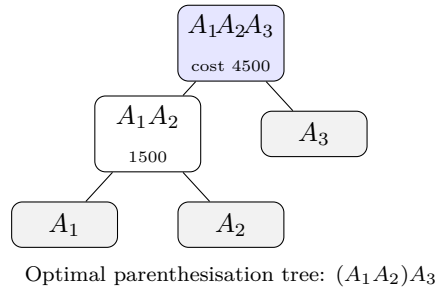
$$m[i][j] = \min_{i \leq k < j} \{m[i][k] + m[k+1][j] + p_{i-1} p_k p_j\}.$$

We also store $s[i][j] = k$ achieving the minimum to reconstruct the parenthesisation. There are $\Theta(n^2)$ subproblems, each trying $O(n)$ splits: $O(n^3)$ time, $O(n^2)$ space. Fill by increasing chain length $l = 2 \dots n$.

Example: $p = (10, 30, 5, 60)$ for $A_1(10 \times 30)$, $A_2(30 \times 5)$, $A_3(5 \times 60)$. Then $m[2][3] = 30 \cdot 5 \cdot 60 = 9000$, $m[1][2] = 10 \cdot 30 \cdot 5 = 1500$, and $m[1][3] = \min\{m[1][1] + m[2][3] + 10 \cdot 30 \cdot 60 = 0 + 9000 + 18000, m[1][2] + m[3][3] + 10 \cdot 5 \cdot 60 =$

$1500 + 0 + 3000\} = 4500$ with $k = 2$, i.e.

$(A_1 A_2) A_3$.



6.5 Coin Change: Unbounded Knapsack

Given coin denominations $d_1 \dots d_k$ (unlimited supply) and amount N , find the minimum number of coins to make N (or report impossible).

Let $dp[x]$ be the minimum coins for amount x , with $dp[0] = 0$ and $dp[x] = \infty$ for $x < 0$ initially. For $x \geq 1$:

$$dp[x] = 1 + \min_{d_i \leq x} dp[x - d_i],$$

taking ∞ if no $d_i \leq x$ yields a finite predecessor. Equivalently, tabulate $x = 1 \dots N$ and for each coin d_i relax $dp[x]$ from $dp[x - d_i]$.

This is unbounded knapsack: each coin may be reused, so the recurrence looks back at the same coin set (contrast 0/1 knapsack which moves to $i - 1$). Time $O(kN)$, space $O(N)$. Reconstruct coins by storing $choice[x]$, the first coin used.

Example: $d = (1, 3, 4)$, $N = 6$. Then dp is 0, 1, 2, 1, 1, 2 for $x = 0 \dots 6$; e.g. $dp[6] = 1 + \min(dp[5], dp[3], dp[2]) = 1 + 1 = 2$ via $3 + 3$. Greedy (always take largest $d_i \leq x$) fails here: it gives $4 + 1 + 1$ (3 coins).

```

1 def coin_change(coins, N):
2     INF = N+1
3     dp = [INF]*(N+1); choice=[-1]*(N+1)
4     dp[0]=0
5     for x in range(1, N+1):
6         for c in coins:
7             if c<=x and dp[x-c]+1 < dp[x]:
8                 dp[x]=dp[x-c]+1; choice[x]=c
9     return dp[N] if dp[N]!=INF else -1 # -1 = impossible
  
```

The same table answers counting variants: number of ways uses $ways[x] += ways[x - d_i]$ with careful ordering to avoid permuting combinations.

6.6 Exercises

1. **LCS by hand.** Compute the full c and b tables for $X = \langle B, D, C, A \rangle$, $Y = \langle A, B, C, D \rangle$. List all distinct LCSs and trace each backtracking path.
2. **Knapsack trace.** For capacity $W = 6$ and items $(w, v) = (1, 1), (2, 4), (3, 4), (4, 5)$, fill the K table, state the optimum value, and backtrack to find every optimal subset. Then give the $O(W)$ -space tabulated code and explain why the inner loop must run backwards.
3. **Matrix chain.** Let $p = (5, 10, 3, 12, 5)$. Compute all $m[i][j]$ and $s[i][j]$ by hand, give the optimal parenthesisation and its cost, and argue why the greedy choice of cheapest single multiplication first is not optimal.
4. **Coin change variants.** With $d = (1, 3, 4)$ and $N = 6$, (a) show the dp table and the coins chosen, (b) give a counterexample where greedy fails for $d = (1, 7, 10)$, and (c) modify the DP to count the number of distinct combinations (order irrelevant) and compute it for $N = 6$.
5. **Edit distance.** Define edit distance (insert, delete, replace cost 1). Derive recurrence for $ed[i][j]$ on prefixes, state table size and filling order, and adapt to output an optimal edit sequence.

Chapter 7

Graph Traversal, Shortest Paths, and String Matching

“Graphs and strings are not different worlds.” BFS discovers distance as surely as KMP discovers pattern: both exploit structure to avoid redundant work.

Prerequisites: Chapter 1 (invariants), Chapter 2 (asymptotics, heaps), and basic graph notation $G = (V, E)$ directed or undirected, $|V| = n$, $|E| = m$.

Learning Outcomes

After this chapter you will be able to:

- (L1) trace BFS/DFS, explain queue/stack roles and analyse $O(n + m)$ time;
- (L2) implement Dijkstra with a binary heap and prove correctness via its set invariant;
- (L3) run Bellman–Ford, detect negative cycles, and derive Floyd–Warshall as DP over k ;
- (L4) compute the KMP prefix function π , simulate its automaton, and analyse Rabin–Karp rolling hash.
- (L5) run Ford–Fulkerson on small networks, explain residual graphs, and state the max-flow min-cut theorem.

7.1 Graph Traversal: BFS and DFS

Let $G = (V, E)$ be unweighted for traversal; weights return in §7.2.

How to store a graph? Think of friends: a *matrix* is a full $n \times n$ attendance sheet (1 if i knows j) — $\Theta(n^2)$ even for a sparse class; a *list* stores only who each person actually knows. We use adjacency lists: for each v keep its neighbour list; scanning all edges costs $\Theta(n + m)$, not $\Theta(n^2)$.

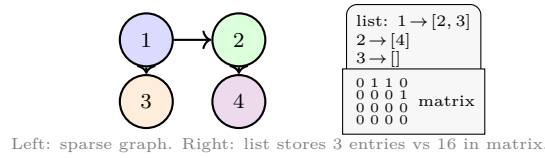


Figure 7.1: Adjacency lists vs matrix — lists pay for edges present, matrices pay for all pairs.

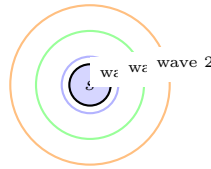
7.1.1 BFS: Queue and Layering

Queue = line, Stack = pile. A *queue* is a supermarket line: first in, first out (FIFO) — you leave in the order you arrived. A *stack* is a pile of plates: last in, first out (LIFO) — you take the top plate. BFS uses a line (fair: all distance-1 vertices before distance-2); DFS uses a pile (commit deep, backtrack). Same graph, swapped container, different order. $\text{BFS}(G, s)$ explores in waves of increasing distance from source s . It uses a FIFO *queue* Q : s is enqueued, then repeatedly dequeued, and each undiscovered neighbour is enqueued.

```

BFS( $G, s$ ):  $\text{dist}[s] \leftarrow 0$ ;  $Q \leftarrow [s]$ ;  $\text{visited} \leftarrow \{s\}$ 
while  $Q \neq \emptyset$ :  $u \leftarrow \text{DEQUEUE}(Q)$ 
for each  $(u, v) \in E$ : if  $v \notin \text{visited}$  then  $\text{dist}[v] \leftarrow \text{dist}[u] + 1$ ;  $\text{parent}[v] \leftarrow u$ ;  $\text{ENQUEUE}(Q, v)$ 

```



Pond ripple: BFS visits all of wave d before wave $d+1$. FIFO queue holds at most two adjacent waves.

Figure 7.2: Wave-front view of BFS — distance layers expand like ripples; the queue is the line of people in ripple order.

The *layering* property: Q always contains vertices of at most two consecutive distances $d, d+1$ from s , all d before $d+1$. Hence $\text{dist}[v]$ is the shortest-path distance $\delta(s, v)$ in an unweighted graph (proved by induction on d). Parent pointers form a *BFS tree* rooted at s ; every non-tree edge connects vertices whose layers differ by at most one.

Example 7.1 (BFS distances on Fig. 7.3). Queue evolution: $[s] \rightarrow [a, b] \rightarrow [b, c, d] \rightarrow [c, d, e] \rightarrow [d, e] \rightarrow [e] \rightarrow []$. Thus $\text{dist}[s] = 0$, $\text{dist}[a] = \text{dist}[b] = 1$, $\text{dist}[c] = \text{dist}[d] = \text{dist}[e] = 2$; parent pointers give shortest-path tree $s \rightarrow a \rightarrow c$, $s \rightarrow \{a, b\} \rightarrow d$, $s \rightarrow b \rightarrow e$. Any $s \rightarrow e$ path needs at least 2 edges, and BFS finds one.

Analysis: each vertex enters Q once and each adjacency list is scanned once, giving $O(n + m)$ time if G is stored as adjacency lists. BFS also tests bipartiteness (2-colour by layer parity; an edge within a layer $\Rightarrow$ odd cycle) and finds connected components by restarting from any unvisited vertex. Multi-source BFS (enqueue all sources at distance 0) solves “nearest source” or “shortest in unweighted grid” problems in the same bound.

7.1.2 DFS: Stack and Structure

DFS replaces the queue by a LIFO *stack* (or recursion): go deep before going wide. Iterative version pushes s onto a stack; at each step pop u and push its undiscovered neighbours.

```

DFS-VISIT( $u$ ): mark  $u$  discovered; for each  $(u, v) \in E$  if  $v$  undiscovered then  $\text{parent}[v] \leftarrow u$ ; DFS-VISIT( $v$ ); mark  $u$  finished.

```

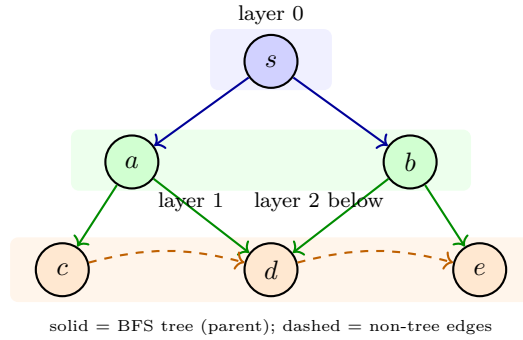


Figure 7.3: BFS layering from s . Queue order visits layer 0, then layer 1 (a, b), then layer 2 (c, d, e). Each vertex is enqueued once; every edge examined once (twice if undirected): $O(n + m)$ time and $O(n)$ extra space.

With discovery/finish times $d[u]$, $f[u]$, DFS classifies edges (undirected: tree/back; directed: tree/back/forward/cross) and yields the *parenthesis theorem*: intervals $[d[u], f[u]]$ are nested or disjoint. This powers cycle detection (back edge $\Leftrightarrow$ cycle in directed graphs), topological sort (decreasing $f[u]$ on a DAG), and Kosaraju’s two-pass SCC algorithm. Like BFS: $O(n + m)$ time, $O(n)$ stack/colour array.

Queue vs. stack intuition: BFS’s queue enforces breadth (fairness across branches); DFS’s stack enforces depth (commitment to one branch). Swapping the container is the only algorithmic difference, yet the resulting visit orders and edge classifications diverge sharply.

7.2 Shortest Paths

Edge weights $w : E \rightarrow \mathbb{R}$. Let $\delta(s, v)$ denote the true shortest-path weight ($-\infty$ if a negative cycle is reachable from s on some $s \rightarrow v$ path, ∞ if v is unreachable). All algorithms below use $\text{RELAX}(u, v)$: if $\text{dist}[v] > \text{dist}[u] + w(u, v)$ then update $\text{dist}[v]$ and $\text{parent}[v] \leftarrow u$. Flow networks reuse the same relax-and-improve idea via residual augmenting paths; see §7.4.

7.2.1 Dijkstra: Greedy with a Heap

Assumption: $w(e) \geq 0$ for all e . Dijkstra grows a set S of settled vertices whose final distances are known, repeatedly extracting the unsettled vertex with minimum tentative distance.

```

DIJKSTRA( $G, w, s$ )
1  $\text{dist}[s] \leftarrow 0$ ;  $\text{dist}[v \neq s] \leftarrow \infty$ ;  $S \leftarrow \emptyset$ ; min-heap  $Q$  on  $\text{dist}$ 
2 while  $Q \neq \emptyset$  do
3    $u \leftarrow \text{EXTRACTMIN}(Q)$ ;  $S \leftarrow S \cup \{u\}$ 
4   for each  $(u, v) \in E$  do  $\text{RELAX}(u, v)$ :
   if  $\text{dist}[v] > \text{dist}[u] + w(u, v)$  then
      $\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$ ;  $\text{DECREASEKEY}(Q, v)$ 

```

Heap = priority line. A queue serves in arrival order; a *heap* serves the smallest key first, like a hospital triage line where the most urgent patient jumps ahead. Dijkstra's Q is triaged by *dist*: EXTRACTMIN takes the currently closest unsettled vertex, DECREASEKEY moves a vertex forward when a shorter route is found. Heap implementation: binary heap gives $O((n + m) \log n)$; Fibonacci heap improves to $O(m + n \log n)$. Early

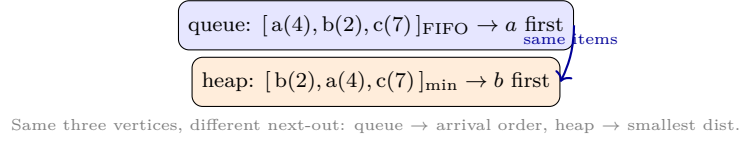


Figure 7.4: Queue vs heap: Dijkstra needs a priority line, not a fair line.

exit when a target t is extracted — its distance is already final. Relaxation is the only place *dist* decreases, mirroring the BFS layering but weighted.

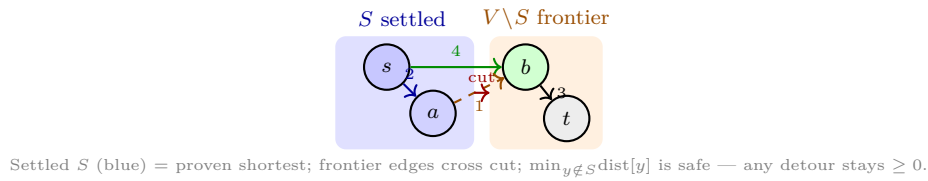


Figure 7.5: Dijkstra frontier: S is the safe zone; the closest frontier vertex cannot be improved via a non-negative detour.

Correctness — invariant: At the start of each iteration with S already settled,

$$(I) \quad \forall x \in S : \text{dist}[x] = \delta(s, x) \quad \text{and} \quad \forall y \notin S : \text{dist}[y] = \min_{P: s \rightarrow y \text{ with inner vertices in } S} w(P).$$

Initialisation: $S = \emptyset$, (I) holds vacuously ($\text{dist}[s] = 0$). *Maintenance:* Let $u \notin S$ have minimum *dist* among $V \setminus S$. Any $s \rightarrow u$ path must leave S at some edge (x, y) with $x \in S$, $y \notin S$; its weight is $\geq \text{dist}[y] \geq \text{dist}[u]$ because all weights are non-negative, so no unseen path beats $\text{dist}[u]$. Extracting u preserves (I) after relaxing its edges. *Termination:* $S = V$ gives $\text{dist}[v] = \delta(s, v)$ for all v . If w has a negative edge the inequality fails and Dijkstra may err — a standard counterexample is $s \xrightarrow{2} b \xrightarrow{-2} a$ where a is settled too early.

7.2.2 Bellman–Ford: Relaxation and Negative Cycles

No non-negativity needed. Relax every edge $n - 1$ times, in any order:

```
for i ← 1 to n - 1 do for each (u, v) ∈ E do RELAX(u, v)
```

After k rounds, $\text{dist}[v]$ equals the shortest weight among $s \rightarrow v$ paths using $\leq k$ edges (induction on k : the k -edge optimum extends a $(k - 1)$ -edge optimum by one relaxation). Since any shortest simple path uses at most $n - 1$ edges, after $n - 1$ rounds $\text{dist} = \delta$ iff no negative cycle is reachable. One extra round detects one: if any relaxation succeeds, a negative cycle exists and $\delta = -\infty$ on its reachable vertices; tracing parent pointers retrieves it.

Example 7.2 (Bellman–Ford detects a negative cycle). Vertices $\{s, a, b\}$ with $s \xrightarrow{1} a$, $a \xrightarrow{1} b$, $b \xrightarrow{-3} a$. After $n - 1 = 2$ rounds $\text{dist}[a] = 1$, $\text{dist}[b] = 2$. Round 3 relaxes $b \rightarrow a$: $\text{dist}[a] > 2 + (-3) = -1$, so $\text{dist}[a]$ drops — negative cycle $a \rightarrow b \rightarrow a$ of weight -2 detected.

Time $O(nm)$, space $O(n)$ — slower than Dijkstra but fully general.

7.2.3 Floyd–Warshall: DP over Intermediate Vertices

For all-pairs shortest paths, let $d^{(k)}[i, j]$ be the shortest $i \rightarrow j$ weight whose *inner* vertices lie in $\{1, \dots, k\}$ (endpoints unrestricted). Then

$$d^{(0)}[i, j] = \begin{cases} 0 & i = j \\ w(i, j) & (i, j) \in E, \\ \infty & \text{else} \end{cases} \quad d^{(k)}[i, j] = \min(d^{(k-1)}[i, j], d^{(k-1)}[i, k] + d^{(k-1)}[k, j]).$$

Iterating $k = 1 \dots n$ in place (a single $n \times n$ matrix suffices since $d^{(k)}[i, k] = d^{(k-1)}[i, k]$ and $d^{(k)}[k, j] = d^{(k-1)}[k, j]$) yields $\delta(i, j) = d^{(n)}[i, j]$. Time $\Theta(n^3)$, space $\Theta(n^2)$. Negative-cycle check: $\exists i : d^{(n)}[i, i] < 0$. Despite cubic cost, Floyd is concise, cache-friendly, and reconstructs paths via a successor matrix $\text{nxt}[i, j]$ updated alongside d .

Remark 7.1 (Choosing among shortest-path algorithms). Non-negative weights and single source: Dijkstra. Negative weights or cycle detection needed: Bellman–Ford. Dense all-pairs or transitive closure: Floyd–Warshall. On sparse graphs, running Dijkstra from each source with a heap can beat Floyd at $O(n(m \log n))$.

7.3 String Matching

Text $T[1 \dots n]$, pattern $P[1 \dots m]$, alphabet Σ ($n \geq m$). Report all shifts s with $T[s + 1 \dots s + m] = P$. We compare three strategies that trade comparison cost for preprocessing.

7.3.1 Naïve $\Theta((n - m + 1)m)$ Scan

Slide P over each alignment $s = 0 \dots n - m$ and compare $P[1 \dots m]$ against $T[s + 1 \dots s + m]$ character by character, breaking on first mismatch. Worst case $\Theta((n - m + 1)m)$ — e.g., $T = a^n$, $P = a^{m-1}b$ forces m comparisons at each of $n - m + 1$ positions. No preprocessing, $O(1)$ extra space. The next two algorithms avoid re-scanning characters already matched.

7.3.2 KMP: Prefix Function and Automaton

Border = overlap. For $P = \text{abab}$, prefix **ab** = suffix **ab** — it *borders* the string. On mismatch you slide the pattern so the longest border that still matches the text suffix stays aligned; you do not restart at 0. Let

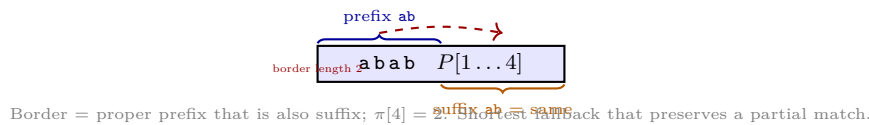


Figure 7.6: Border picture for **abab** — the overlap that lets KMP fall back instead of restarting.

$\pi[q] = \max\{k < q : P[1 \dots k] \text{ is suffix of } P[1 \dots q]\}$ for $1 \leq q \leq m$ ($\pi[1] = 0$); equivalently $\pi[q]$ is the longest proper border of the prefix $P[1 \dots q]$. Computed in $O(m)$ by amortised fallback:

```

 $\pi[1] \leftarrow 0; k \leftarrow 0; \text{for } q \leftarrow 2 \text{ to } m: \text{ while } k > 0 \wedge P[q] \neq P[k+1] \text{ do } k \leftarrow \pi[k]; \text{ if } P[q] = P[k+1] \text{ then } k \leftarrow k+1;$ 
 $\pi[q] \leftarrow k$ 
    
```

Matching scans T once, maintaining $q = \text{length of longest prefix of } P \text{ that is a suffix of } T[1 \dots i]$; on mismatch fall back $q \leftarrow \pi[q]$. Total $O(n + m)$ time, $O(m)$ extra space; each character of T is examined at most twice.

The automaton view: state q means “first q characters of P just matched.” Transition $\delta(q, c)$ is the longest prefix of P that is a suffix of $P[1 \dots q]c$; missing edges follow π -links (failure function).

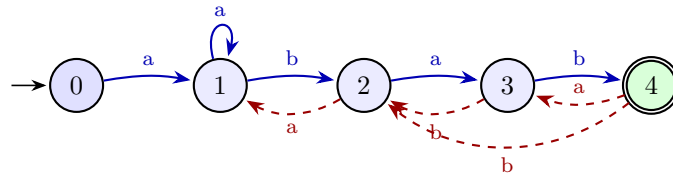


Figure 7.7: KMP automaton for $P = \text{abab}$ ($m = 4$). Solid spine spells P ; failure links (merged into δ) redirect on mismatch via π : $\pi = [0, 0, 1, 2]$. Scanning T never moves backwards, giving $O(n + m)$ time.

7.3.3 Rabin–Karp: Rolling Hash

Clock arithmetic. Working mod q is like a clock with q hours: after $q - 1$ you wrap to 0. Adding or multiplying then taking mod q discards full laps but preserves remainders; collisions are two different strings landing on the same hour mark. A large prime q makes accidental collisions rare. Treat strings as base- b numbers modulo a prime q . Let $h(s) = T[s + 1 \dots s + m] \bmod q$ and $p = P \bmod q$. With precomputed $b^{m-1} \bmod q$,

$$h(s + 1) = (b \cdot (h(s) - T[s + 1] \cdot b^{m-1}) + T[s + m + 1]) \bmod q,$$

so each shift is $O(1)$ after $O(m)$ preprocessing. If $h(s) \neq p$ there is no match; if equal, verify by direct comparison (handles spurious hits from collisions). Expected $O(n + m)$ with good q and $O(nm)$ worst case when every window collides.

Example 7.3 (Rolling hash by hand). Let $b = 10$, $q = 13$ (small for illustration), $T = 31415$, $m = 2$. Then $h(0) = 31 \bmod 13 = 5$, $h(1) = (10 \cdot (5 - 3 \cdot 10) + 4) \bmod 13 = 1$ which equals $14 \bmod 13 = 1$; $p = 15 \bmod 13 = 2$, so neither window matches $P = 15$. With real $b = 256$, $q \approx 10^9 + 7$ and Horner evaluation, the same recurrence applies to strings over bytes.

Double hashing (or $q \gg n$) makes collisions negligible; one text scan then serves 2-D and multi-pattern search such as plagiarism detection over a corpus.

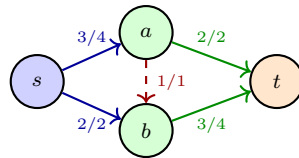
7.4 Maximum Flow: Ford–Fulkerson and Min Cut

Pipes and water. Picture each directed edge as a pipe stamped with its capacity: how much water per second it can carry from a source s to a sink t . A *flow* assigns a rate to every pipe so that no pipe bursts (flow $\leq$ capacity) and every junction except s and t passes on exactly what it receives (inflow = outflow). The goal is to push as much water as possible from s to t .

Definition 7.1 (Flow network and flow). A flow network is a directed graph with capacities $c(u, v) \geq 0$ (0 if no edge), source s and sink t . A flow f obeys $0 \leq f(u, v) \leq c(u, v)$ and conservation $\sum_u f(u, v) = \sum_w f(v, w)$ for all $v \notin \{s, t\}$; its value is $|f| = \sum_v f(s, v)$.

Residual graph: the room that is left. Given f , the *residual capacity* $c_f(u, v) = c(u, v) - f(u, v) + f(v, u)$ says how much more can still go $u \rightarrow v$: spare room forward, plus already-sent flow that could be cancelled backwards. Edges with $c_f > 0$ form the residual graph G_f ; any $s \rightarrow t$ path P in G_f (an *augmenting path*) carries bottleneck $\Delta = \min_{(u,v) \in P} c_f(u, v)$ extra units.

FORD-FULKERSON: $f \leftarrow 0$; while an $s \rightarrow t$ path P exists in G_f do augment f along P by bottleneck $\Delta = \min c_f$.



Labels flow/capacity. Dashed $a \rightarrow b$ saturated; residual back-edge $b \rightarrow a$ (capacity 1) lets later paths reroute. Cut $\{s, a\}/\{b, t\}$ has capacity $2 + 2 + 1 = 5 = |f|$.

Figure 7.8: A flow of value 5 with residual back-edge; the dashed cut certifies optimality via max-flow min-cut.

Example 7.4 (Three augmentations). Capacities as in Fig. 7.8. Augment $s \rightarrow a \rightarrow t$ by 2 (bottleneck $a \rightarrow t$), then $s \rightarrow b \rightarrow t$ by 2, then $s \rightarrow a \rightarrow b \rightarrow t$ by 1 through $a \rightarrow b$. Now the residual reach of s is $\{s, a\}$ only, so no augmenting path remains and $|f| = 2 + 2 + 1 = 5$ is maximum.

Theorem 7.1 (Max-flow min-cut). For every network, the maximum flow value equals the minimum capacity $\sum_{u \in S, v \notin S} c(u, v)$ over all s - t cuts $(S, V \setminus S)$. Ford–Fulkerson terminates with a maximum flow for integer capacities in $O(m \cdot |f^*|)$ time; always augmenting along a shortest path (Edmonds–Karp BFS) costs $O(nm^2)$ and works for irrational capacities too.

7.5 Chapter Notes

BFS (Moore, 1959) and DFS (Tarjan, 1972) in modern form underpin all graph algorithms. Dijkstra (1959) with binary heaps is the practical default; Bellman–Ford (1958) and Floyd–Warshall (1962) handle negative weights. KMP (Knuth–Morris–Pratt, 1977) and Rabin–Karp (1987) remain textbook representatives of automaton vs. hashing; maximum flows and minimum cuts are CLRS Ch. 26. CLRS Chs. 20–24, 32 covers all topics in depth.

7.6 Exercises

E7.1 BFS layering vs. DFS depth. Run BFS and DFS (alphabetical adjacency) from s on the graph of Fig. 7.3 with extra edge $e \rightarrow a$. List discovery order, $\text{dist}[\cdot]$ from BFS, and DFS finish times $f[\cdot]$. Give a graph where DFS from s never discovers a vertex that BFS does. Can that happen if the graph is undirected and connected? Justify.

E7.2 Dijkstra proof in action. (a) Execute Dijkstra with a binary heap from s on: $s \xrightarrow{4} a$, $s \xrightarrow{2} b$, $b \xrightarrow{1} a$, $a \xrightarrow{3} t$, $b \xrightarrow{5} t$. Show heap contents per extraction and invariant (I). (b) Add edge $b \xrightarrow{-3} a$ and show extraction

order violates (I) and yields a wrong distance. (c) Would Bellman–Ford recover the correct distances and how many rounds suffice?

E7.3 KMP prefix function. (a) Compute π for $P = \mathbf{aabaaab}$ step by step. (b) Draw the KMP automaton states $0 \dots 7$ and label failure (π) links. (c) Trace KMP on $T = \mathbf{aabaabaaab}$ and count character comparisons versus naïve scan. Prove $O(n + m)$ is tight by giving a family attaining $\Theta(n + m)$ comparisons.

E7.4 Rolling hash design. Let $b = 256$, $q = 101$. (a) Compute the Rabin–Karp hash of $P = \mathbf{abc}$ and the rolling update to the next window in $T = \mathbf{abcd}$. (b) Exhibit $P \neq Q$ over $\{a, b\}$ colliding modulo 101, and explain why a second modulus q_2 reduces collision probability to $\approx 1/(q q_2)$. (c) Argue expected time is $O(n + m)$ when $q \geq m^2$ and verification is $O(m)$, assuming uniform hashing.

E7.5 Max flow and min cut. (a) Starting from zero flow on Fig. 7.8, list the residual graph G_f and bottleneck Δ for each of the three augmentations in the worked example. (b) Verify cut $\{s, a\}/\{b, t\}$ has capacity 5 and prove no flow can exceed any s – t cut capacity. (c) Give integer capacities where a poor augmenting-path choice needs ≥ 10 augmentations while Edmonds–Karp (BFS paths) finishes in 2; state both runtimes.

Chapter 8

NP-Completeness

“If $P = NP$, the world would change profoundly; if $P \neq NP$, we must cope.” NP-completeness tells us which problems admit efficient algorithms and which almost certainly do not.

Prerequisites: Asymptotic analysis, polynomial time, reductions, and graph basics (VERTEX COVER, CLIQUE).

Learning Outcomes

After this chapter you will be able to:

- (L1) distinguish P, NP, coNP via verifiers and certificates;
- (L2) define polynomial reduction and NP-completeness (Cook–Levin) and chain reductions $SAT \rightarrow 3SAT \rightarrow CLIQUE \rightarrow VERTEX\ COVER$;
- (L3) prove the 2-approximation for VERTEX COVER and bound greedy SET COVER;
- (L4) discuss P vs. NP and coping strategies for NP-complete problems.

8.1 P, NP, and coNP

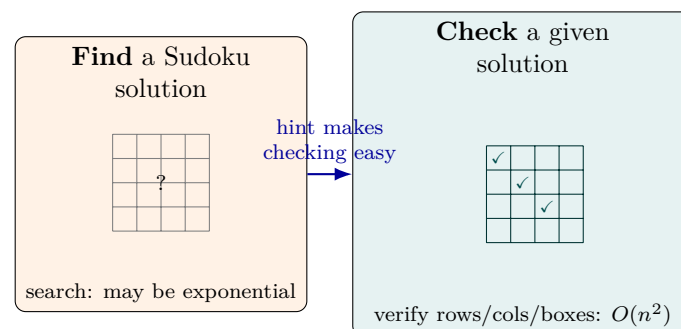


Figure 8.1: Finding vs. checking. Sudoku captures P vs. NP: *finding* a solution is hard, *checking* a claimed solution (certificate) is easy. P = can find quickly; NP = can check quickly given a hint.

Intuition first: *find* vs. *check*. Completing an empty Sudoku can take exhaustive search, but checking a filled grid is routine (scan rows, columns, boxes). NP formalises this: even if finding a proof is hard, a hint (certificate) makes verification fast. Fig. 8.1 is the mental model for the definitions below.

A language $L \subseteq \{0,1\}^*$ is a decision problem: $x \in L$ means “yes”. We measure time in $n = |x|$ on a deterministic RAM/Turing model.

Definition 8.1 (P, informal). P is the class of languages decidable by a deterministic algorithm in polynomial time, i.e. $O(n^c)$ for some constant c .

Think: sorting, BFS, Dijkstra are in P—routine even for large n when c is small.

Definition 8.2 (NP, verifier view). $L \in \text{NP}$ if there exists a deterministic polynomial-time *verifier* $V(x, c)$ and polynomial p such that $x \in L \iff \exists c$ with $|c| \leq p(|x|)$ and $V(x, c) = 1$. The string c is the *certificate* (witness): it proves membership and can be checked quickly, even if finding it is hard.

Equivalently, L is decided by a nondeterministic machine that guesses c then verifies. Example: 3SAT certificate is a satisfying assignment; CLIQUE certificate is the k vertices. Clearly $\text{P} \subseteq \text{NP}$: ignore the certificate and decide directly.

Remark 8.1 (coNP). $\text{coNP} = \{\bar{L} : L \in \text{NP}\}$: languages whose “no”-answers have short certificates. Example: UNSAT is in coNP but not known in NP. We have $\text{P} \subseteq \text{NP} \cap \text{coNP}$; whether $\text{NP} = \text{coNP}$ is open.

Example 8.1 (Certificates at work). For $\text{CLIQUE}(G, k)$, list k vertices; check $\binom{k}{2}$ edges in $O(n^2)$. For $\text{SUBSET SUM}(S, t)$, the subset itself. Verification is easy while search appears hard—the essence of NP.

8.2 Polynomial Reductions and NP-Completeness

Definition 8.3 (Polynomial reduction). $A \leq_p B$ if there is polynomial-time f with $x \in A \iff f(x) \in B$ for all x . Intuition: B is at least as hard as A ; a solver for B solves A with one call plus cost of f .

Definition 8.4 (NP-complete, NP-hard). L is NP-hard if every $A \in \text{NP}$ satisfies $A \leq_p L$. L is NP-complete if $L \in \text{NP}$ and L is NP-hard. A polynomial algorithm for one NP-complete problem yields one for all.

Theorem 8.1 (Cook–Levin, stated without proof). SAT is NP-complete.

Every NP verification can be encoded as a Boolean tableau whose satisfiability mirrors existence of a certificate. Since $\leq_p$ is transitive, it suffices to reduce from any known NP-complete problem.

8.2.1 SAT $\leq_p$ 3SAT: clause splitting

A SAT clause $(l_1 \vee \dots \vee l_k)$ with $k > 3$ is split using fresh y_i (Fig. 8.2):

$$(l_1 \vee l_2 \vee y_1) \wedge (\bar{y}_1 \vee l_3 \vee y_2) \wedge \dots \wedge (\bar{y}_{k-3} \vee l_{k-1} \vee l_k).$$

Clauses with 1 or 2 literals are padded by repeating a literal. The 3-CNF is satisfiable iff the original is, and $|f(x)| = O(|x|)$. Hence $\text{SAT} \leq_p \text{3SAT}$ and 3SAT is NP-complete.

Example 8.2 (Clause splitting). $(x_1 \vee \bar{x}_2 \vee x_3 \vee x_4)$ becomes $(x_1 \vee \bar{x}_2 \vee y_1) \wedge (\bar{y}_1 \vee x_3 \vee y_2) \wedge (\bar{y}_2 \vee x_4 \vee x_4)$.

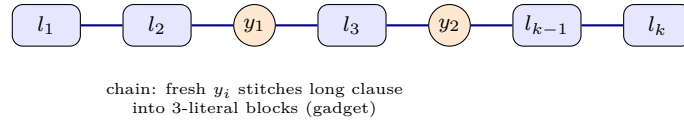


Figure 8.2: SAT→3SAT splitting gadget: a k -clause becomes $k-2$ clauses of size 3 using $k-3$ fresh variables. The gadget is satisfiable iff the original clause is.

8.2.2 3SAT $\leq_p$ Clique: triangles and conflicts

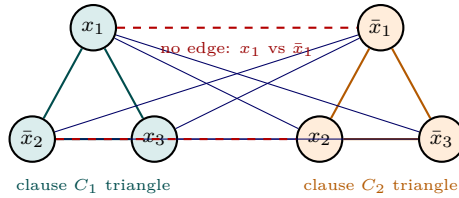


Figure 8.3: 3SAT→CLIQUE gadget: one triangle per clause; connect compatible literals across clauses (omit conflict edges $x-\bar{x}$). An m -clique picks one literal per clause with no conflicts iff ϕ is satisfiable.

Given 3-CNF $\phi = C_1 \wedge \dots \wedge C_m$, $C_j = (\ell_{j1} \vee \ell_{j2} \vee \ell_{j3})$, build G with $3m$ vertices—a triangle per clause (Fig. 8.3). Connect compatible literals across clauses: add an edge between vertices in different triangles iff their literals are not complementary (i.e., not x vs. $\bar{x}$). No edges within different triangles beyond that; triangle edges stay. Set $k = m$. Then G has a k -clique iff ϕ is satisfiable. Size is $O(m^2)$, polynomial.

8.2.3 Clique $\leq_p$ Vertex Cover: complement

For $G = (V, E)$, $\bar{G} = (V, \bar{E})$ is the complement. S is a clique in G iff $V \setminus S$ is a vertex cover in $\bar{G}$. So $(G, k) \mapsto (\bar{G}, |V| - k)$ is a reduction, and VERTEX COVER is NP-complete. Chaining further, SET COVER, HAMILTONIAN CYCLE, and SUBSET SUM are also NP-complete.

Remark 8.2 (Hard vs. complete). NP-complete $\subseteq$ NP; NP-hard may lie outside (e.g. halting, optimisation versions). Showing $L \in$ NP is half the proof.

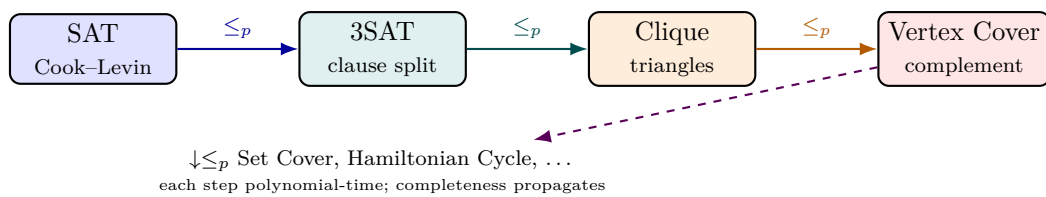


Figure 8.4: Reduction chain from Cook–Levin. Hardness flows rightward.

8.3 Coping with NP-Completeness

When a problem is NP-complete we do not expect an exact polynomial algorithm: approximate, parameterise, or use heuristics.

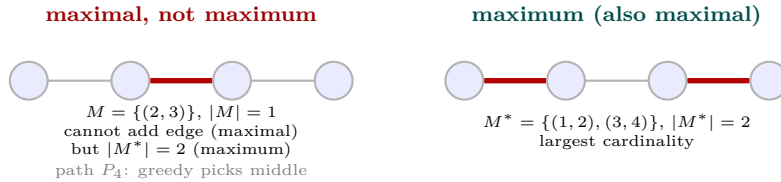


Figure 8.5: Maximal vs. maximum matching. *Maximal*: cannot add any edge (local). *Maximum*: largest possible (global). Approx-VC needs only maximal—any maximal matching gives the 2-bound, no need to compute a maximum matching.

8.3.1 2-approximation for Vertex Cover via maximal matching

APPROX-VC: pick any maximal matching M (greedily add disjoint edges until stuck; Fig. 8.5) and return $C =$ all endpoints of M .

Theorem 8.2. C is a vertex cover with $|C| \leq 2 \cdot \text{OPT}$.

Proof. M maximal $\Rightarrow$ every edge shares an endpoint with some $e \in M$; thus C covers all edges. Any cover needs ≥ 1 vertex per edge of M , disjoint $\Rightarrow \text{OPT} \geq |M|$. But $|C| = 2|M| \leq 2 \cdot \text{OPT}$. Tight on $K_{n,n}$ ($|C| = 2n$, $\text{OPT} = n$). Time $O(n + m)$: scan edges once. Maximal $\neq$ maximum—we do not need a maximum matching (Fig. 8.5).

8.3.2 Greedy Set Cover: $\ln n$ approximation

Repeatedly pick the set covering the most uncovered elements. Standard charging gives $|\text{GREEDY}| \leq H_n \cdot \text{OPT} = O(\log n) \cdot \text{OPT}$ where $H_n = 1 + \frac{1}{2} + \dots + \frac{1}{n}$, essentially optimal unless $P = NP$.

8.3.3 Heuristic for TSP: nearest neighbour

Start at any city, go to nearest unvisited city, return. No constant-factor guarantee for general TSP (unless $P = NP$); on metric instances it can be $\Theta(\log n)$ off, but $O(n^2)$ and often reasonable. Better guarantees need Christofides (1.5-approx on metric TSP).

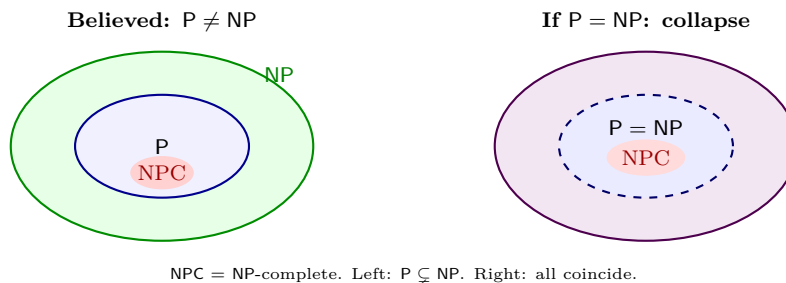


Figure 8.6: Two views of P vs. NP . Left is widely believed; right would make every NP problem tractable.

8.4 P vs. NP and Why $P \neq NP$ Is Believed

P vs. NP asks whether every efficiently verifiable problem is efficiently solvable. $P = NP$ would give polynomial algorithms for all NP-complete problems (cryptography breaks); $P \neq NP$ confirms hardness. Evidence for $P \neq NP$: decades of failed search, oracle/barrier results, and cryptographic practice that would otherwise collapse. A polynomial algorithm for any NP-complete problem propagates via $\leq_p$ to all of NP (Fig. 8.4). Belief is not proof—separation must rule out *all* polynomial algorithms.

Reduction size check. Each construction is polynomial: $SAT \rightarrow 3SAT$ blows k -clause into $k-2$ clauses with $k-3$ fresh variables; $3SAT \rightarrow CLIQUE$ creates $3m$ vertices, $O(m^2)$ edges; $CLIQUE \rightarrow VERTEX\ COVER$ complements in $O(n^2)$.

8.5 Chapter Notes

Cook (1971) and Levin (1973) founded NP-completeness; Karp (1972) listed 21 reductions. Garey–Johnson remains the catalogue. CLRS Ch. 34 is concise; Sipser Ch. 7 gives the language view; Arora–Barak Ch. 2 covers Cook–Levin. Exam tip: to prove X NP-complete, show (a) $X \in NP$ (verifier) and (b) $Y \leq_p X$ for known NP-complete Y .

8.6 Exercises

Exercise 8.1 (Certificate vs. decision). Give verifiers for SUBSET SUM and HAMILTONIAN CYCLE to show each is in NP. Why is the complement not obviously in NP (what about coNP)?

Exercise 8.2 (Reductions by hand). (a) Convert $\phi = (x_1 \vee x_2) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee x_3) \wedge (x_1 \vee \bar{x}_3 \vee x_4 \vee \bar{x}_5)$ to 3-CNF via clause splitting; count new variables/clauses. (b) Apply $3SAT \rightarrow CLIQUE$ to your 3-CNF and draw G ; identify the m -clique for $x_1 = 1, x_2 = 0, x_3 = 1$. (c) Exhibit the complementary vertex cover.

Exercise 8.3 (Approximation analysis). (a) Show 2-approx for APPROX-VC is tight ($|C| = 2 \cdot OPT$ family). (b) Show greedy SET COVER achieves H_n by charging $1/k$. (c) Give 4-city TSP where nearest-neighbour is $> 2\times$ optimal; why no constant factor for general TSP unless $P = NP$?

Exercise 8.4 (P vs. NP consequences). Assume $P = NP$. (a) Show every NP-complete language is in P and outline solving 3SAT in polynomial time. (b) Why would factoring-based crypto be at risk even though factoring is not known NP-complete? (c) If $NP \neq coNP$, what about P vs. NP?